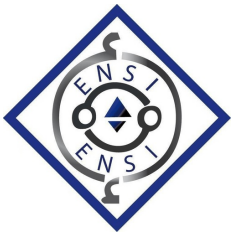
Ministère de l'Enseignement Supérieur

de la Recherche Scientifique et De la Technologie

****

Université de la Manouba

Ecole Nationale Des Sciences De L'Informatique



Projet de Fin d'études

Conception, réalisation et mise en place d'une application pour la gestion des projets



Réalisé par :

Akram Barraj

Encadré par :

Mr. Abdelmonam Kouka

Année Universitaire 2013-2014

Appréciations des encadrants

Mr. Abdelmonam Kouka



Dr. Hamadi Hasni



Dédicaces

A toutes les personnes chères à mon coeur,

A ma très chère maman Selma pour tout son amour, ses sacrifices et son dévouement,

A mon très cher papa Mouldi pour son amour, son soutien et ses encouragements,

A mes chères soeurs Ajmiya et Abir pour leur amour et leur précieuse présence,

A mes chère tantes et oncles,

A toute ma famille et tous mes amis et tous ceux qui n'ont jamais cessé de croire en moi,

Je dédie ce modeste travail, le fruit de mes efforts et de longues années d'études,

Qu'ils y trouvent le couronnement de leurs assistances et l'expression profonde de ma gratitude.

Akram.

Remerciements

Au terme de mon projet de fin d'études, je tiens à adresser mes plus vifs remerciements à toutes les personnes qui, de près ou de loin, ont contribué à l'aboutissement de ce travail dans les meilleures conditions.
Je m'adresse en premier lieu aux membres de l'honorable jury que je remercie d'avoir accepté d'examiner ce rapport.
Je tiens à présenter mes reconnaissances et mes remerciements à mon encadrante à l'ENSI **Dr. Hamadi Hasni**, pour le temps consacré à la lecture et aux réunions qui ont rythmées les étapes de mon projet de fin d'étude. Je la remercie aussi pour sa disponibilité à encadrer ce travail à travers ses critiques et ses propositions d'amélioration.
Un remerciement particulier à mon encadreur dans la société TAC-TIC **Mr. Abdelmonam Kouka**, pour le soutien qu'il m'a apporté tout au long du stage réalisé au sein de cette entreprise.
Enfin je remercie toutes les personnes qui ont contribué de près ou de loin à la réalisation de ce projet de fin d'étude, ainsi qu'au bon déroulement du stage, et dont les noms ne figurent pas dans ce document.

Akram BARRAJ

Résumé

Le présent rapport a été élaboré dans le cadre du projet de fin d'études pour l'obtention du diplôme d'ingénieur en informatique. Le travail est réalisé pendant la période de Février 2014 to Juin 2014.

Il a été réalisé dans TAC-TIC Société et au cours de cette période, nous avons essayé d'être fidèle aux exigences mentionnées dans le document de spécification.

Ce projet consiste à concevoir et à réaliser une application pour la gestion des projets et des feuilles de temps. Cette application couvre la gestion des utilisateurs, des projets, des feuilles de temps et de la messagerie interne.

À cette fin, nous avons été amenés à la compréhension de certains solution proposée, pour trouver et concevoir celui qui convient, à développer les modules nécessaires et enfin de déployer l'application dans un des serveurs TAC-TIC.

Mots clés : Scrum, framework Symfony2, JavaScript, gestion de projet.

Abstract

This project is a part of the project graduation to obtain a computer engineering degree. The work is done during the period of February 2014 to June 2014.

It was realized within TAC-TIC Corporation and during this period we have tried to be faithful to the requirements mentioned in the specification document.
This project is to design and implement an application for projects and timesheets management. This application covers the management of users, projects, timesheets and internal messaging.

For that purpose, we were brought to the understanding of some proposed solution, to find and conceive the appropriate one, to develop necessary modules and finally to deploy the application within one of TAC-TIC servers.

Key words : Scrum, framework Symfony2, JavaScript, project management.

الملخّص

تم تنفيذ هذا العمل في إطار مشروع التخرج للحصول على شهادة الهندسة في علوم الكمبيوتر .ويتم العمل خلال الفترة من فبراير 2014 حتى يونيو عام 2014.

وقد نفذ ضمن داخل شركة TIC-TAC وخلال هذه الفترة حاولنا أن تكون أوفياء للمتطلبات المذكورة في وثيقة المواصفات.

هذا المشروع هو تصميم وتنفيذ تطبيق لإدارة المشاريع و الجداول الزمنية. يغطي هذا التطبيق إدارة المستخدمين والمشاريع والجداول الزمنية والرسائل الداخلية .

لهذا الغرض، قمنا بفهم بعض الحلول المقترحة، لإيجاد وتصور الحل المناسب، بتطوير وحدات الضرورية وأخيرا إلى نشر التطبيق ضمن واحدة من خوادم TIG TAC.

الكلمات المفاتيح: الخلية, سكروم, الاطار سمفوني2, جافا سكريبت, إدارة المشاريع.

Table des matières

Introduction générale **2**

1 Présentation du cadre du projet **3**

1.1 Présentation de l'organisme d'accueil 3

1.1.1 Fiche d'identité 3

1.1.2 Secteurs d'activités 3

1.2 Présentation du sujet 4

1.3 Choix méthodologique 4

1.3.1 Les méthodes agiles 5

1.3.2 Un comparatif des méthodes agiles 5

1.3.3 La méthodologie Scrum 7

1.3.3.1 Introduction à la méthodologie Scrum 7

1.3.3.2 Le choix de la méthodologie Scrum 7

1.3.4 Formalisme adoptée 9

2 Etat de l'art et étude de l'existant **11**

2.1 Situation Actuelle 11

2.1.1 Dotproject 11

2.1.2 Ehour 12

2.1.3 Les Limites des solutions existantes 14

2.2 Les Solutions disponibles 14

2.2.1 Les critères de recherche 14

2.2.2 Quelques plateformes étudiées 15

2.2.2.1 ProjeQtOr 15

2.2.2.2 GanttProject 15

2.2.2.3 Redmine 16
2.2.3 Autre solutions 16
2.3 Solution Envisagé 17

3 Spécification et analyse des besoins 18
3.1 Étude des besoins 18
3.1.1 Les acteurs du système 18
3.1.2 Besoins fonctionnels 19
3.1.2.1 Les besoins fonctionnels pour un administrateur 19
3.1.2.2 Les besoins fonctionnels pour les employés 20
3.1.2.3 Les besoins fonctionnels pour les clients 20
3.1.3 Besoins non fonctionnels 21
3.2 Les diagrammes de cas d'utilisations 21
3.2.1 Le cas d'utilisation général 22
3.2.2 Les cas d'utilisation de l'administrateur 23
3.2.3 Les cas d'utilisation du chef de projet 24
3.2.4 Les cas d'utilisation du développeur 27
3.2.5 Les cas d'utilisation du client 29
3.3 Diagrammes de séquence 30
3.3.1 Diagramme de séquence pour le scénario d'authentification 30
3.3.2 Diagramme de séquence pour le scénario "ajouter employé" 33
3.3.3 Diagramme de séquence pour le scénario gérer projets 34
3.3.4 Diagramme de séquence pour le scénario d'activation d'un compte 34
3.3.5 Diagramme de séquence pour le scénario de recherche 35
3.3.6 Diagramme de séquence pour le scénario d'ajout d'une nouvelle tache . . 37
3.3.7 Diagramme de séquence pour le scénario de validation d'un ticket 37

4 Conception 39
4.1 Conception générale 39
4.1.1 Conception architecturale 39
4.1.1.1 L'architecture trois-tiers 40
4.1.1.2 Le modèle MVC 41
4.1.1.3 Architecture retenue 42

4.2 Conception détaillée . . . 43
4.2.1 Diagramme de paquetage . . . 43
4.2.2 Diagramme de classes . . . 44
4.2.2.1 Le paquetage « Utilisateur » . . . 44
4.2.2.2 Le paquetage « Projet » . . . 46
4.2.2.3 Le paquetage « Évènement » . . . 48
4.2.2.4 Le paquetage « Timesheets » . . . 49
4.2.2.5 Paquetage « Messagerie » . . . 49
4.2.3 Diagrammes de séquences d'objets . . . 50
4.2.3.1 Diagramme de séquences pour le scénario d'authentification . . 51
4.2.3.2 Diagramme de séquences pour le scénario d'ajout d'un nouveau gestionnaire . . . 51
4.2.3.3 Diagramme de séquences pour le scénario d'ajout d'un nouveau projet . . . 52
4.2.4 Conception de la base de données . . . 53

5 Réalisation **56**
5.1 Réalisation par rapport au backlog . . . 56
5.1.1 Le déroulement des sprints . . . 56
5.1.2 Facteur de disponibilité . . . 57
5.2 Environnement de travail . . . 57
5.2.1 Environnement matériel . . . 57
5.2.2 Choix du framework de développement . . . 57
5.2.2.1 Les frameworks PHP les plus connus . . . 57
5.2.2.2 Le framwork de développement Symfony 2 . . . 58
5.2.3 Environnement logiciel . . . 59
5.2.3.1 **La modélisation UML** . . . 59
5.2.3.2 **IDE Eclipse Kepler 4.3** . . . 59
5.2.3.3 Autres outils . . . 59
5.3 Travail réalisé . . . 60
5.3.1 Interface d'authentification . . . 60
5.3.2 Interface d'accueil . . . 61
5.3.3 Interfaces tous les projets . . . 62

5.3.4 Interface Ajout d'un projet 62
5.3.5 Interfaces suivre un projet 63
5.3.6 Interfaces diagramme de gantt 65
5.3.7 Interfaces de messagerie 65
5.3.8 Interfaces du calendrier 66
5.3.9 Interfaces suivi d'un ticket 67
5.3.10 Interfaces statistiques 67
5.4 Chronogramme 68

Conclusion générale **70**

A L'architecture de Symfony2 **73**

Table des figures

1.1 Les activités de TAC-TIC 4
1.2 Schéma illustratif de SCRUM 8

2.1 DotProject : Tableau de bord d'un utilisateur 12
2.2 Ehour : Génération des rapports et des statistiques 13
2.3 Apperçu de GanttProject 16

3.1 Les différents utilisateurs de l'application 19
3.2 Diagramme de cas d'utilisation général 22
3.3 Diagramme de cas d'utilisation général :Administrateur 23
3.4 Diagramme de cas d'utilisation : administrateur/Gérer profils 24
3.5 Diagramme de cas d'utilisation général : Chef de projet 25
3.6 Diagramme de cas d'utilisation : chef de projet/Gérer projets 26
3.7 Diagramme de cas d'utilisation : chef de projet/Gerer messages 27
3.8 Diagramme de cas d'utilisation général : Développeur 28
3.9 Diagramme de cas d'utilisation : Développeur/Gerer calendrier 28
3.10 Diagramme de cas d'utilisation général : Client 29
3.11 Diagramme de cas d'utilisation : Client/Gerer tickets 30
3.12 Diagramme de séquence pour le scénario d'authentification 31
3.13 Diagramme de séquence pour le scénario renouveler mot de passe 32
3.14 Diagramme de séquence pour le scénario ajouter un nouveau compte "employé" 33
3.15 Diagramme de séquence pour le scénario gérer clients 34
3.16 Diagramme de séquence pour le scénario activer un compte "client" 35
3.17 Diagramme de séquence pour le scénario recherche projet 36
3.18 Diagramme de séquence pour le scénario ajout nouvelle tache 37

3.19 Diagramme de séquence pour le scénario de validation d'un ticket 38

4.1 Architecture trois-tiers 41
4.2 L'interaction entre le modèle, la vue et le contrôleur 42
4.3 Diagramme de paquetage 43
4.4 Diagramme de classe pour le paquetage "Utilisateur" 45
4.5 Diagramme de classe pour le paquetage "Projet" 47
4.6 Diagramme de classe pour le paquetage "Évènement" 48
4.7 Diagramme de classe pour le paquetage "Timesheets" 49
4.8 Diagramme de classe pour le paquetage "Messagerie" 50
4.9 Diagramme de séquence d'authentification 51
4.10 Diagramme de séquence pour l'ajout d'un employé 52
4.11 Diagramme de séquence pour l'ajout d'un projet 52
4.12 Vue globale de la base de données 53
4.13 Modèle détaillé de la base de données 54

5.1 Interface d'authentification 61
5.2 Page d'accueil pour un chef de projet 61
5.3 Tous les projets 62
5.4 Ajout d'un projet 63
5.5 Page détails d'un projet 64
5.6 Page du diagramme de gantt 65
5.7 Envoyer un message 66
5.8 Le calendrier personnel 66
5.9 Suivi d'un ticket 67
5.10 statistiques sur la réalisation des projets/employés 68
5.11 Le chronogramme du travail 69

A.1 L'arborescence d'une application Symfony2 73
A.2 Schématisation du flux d'exécution d'une requête par Symfony 76
A.3 Le toolbare de débougage 77

Liste des tableaux

1.1 Tableau comparatif de quelques méthodes agiles 6

5.1 Les frameworks PHP les plus connus 58

Introduction

L'engouement récent des technologies de l'information et de la communication dans le monde force les organisations à s'intéresser au système d'information. Il est devenu un point central dans leur développement

L'organisation devient de plus en plus dépendante des solutions informatiques choisies. Ces choix influent directement le comportement dynamique ou non de l'entreprise, et autorisent, par la même, des réactions rapides et adaptées au changement de l'environnement technique, économique, social... .

Le système d'information est donc un support d'aide à la décision qui permet d'améliorer l'efficacité et la qualité des décisions prises, un outil de travail coopératif autour de données consolidées et correctement partagées.

Ce qui préoccupe les entrepreneurs c'est l'amélioration de l'efficacité et ses niveaux de productivité, ainsi l'augmentation du chiffre d'affaires qui se base sur le bon choix des projets et l'excellence dans la réalisation en utilisant les ressources existante de façon efficace.

C'est dans ce cadre se situe notre projet de fin d'étude, la société TACTIC nous a confié le développement d'une application web pour la gestion des projets et la suivie des feuilles des temps (conception, développement et mise en place).

Dans le présent document, organisé en cinq chapitres, nous commencerons par introduire le contexte général du projet, à savoir l'organisme d'accueil ainsi que le contexte du projet. Ensuite, nous exposerons l'étude préalable que nous avons menée durant ce stage. Puis nous enchaînerons par la spécification des besoins de notre application. Au quatrième chapitre, nous

détaillerons l'approche adoptée dans l'étape de conception. Et pour clôturer nous décrirons les étapes de réalisation, les outils utilisés ainsi que les résultats obtenus à travers quelques interfaces de l'application.

Chapitre 1

Présentation du cadre du projet

Introduction

Ce chapitre a pour objectif de situer notre projet dans son contexte général. Ainsi, nous commençons par la présentation de l'organisme d'accueil. Ensuite, nous décrivons brièvement le sujet, les objectifs à atteindre, et la méthodologie de travail adoptée.

1.1 Présentation de l'organisme d'accueil

Le présent projet est réalisé dans le cadre de la préparation d'un mémoire de fin d'études présenté en vue de l'obtention du diplôme d'ingénieur en informatique à l'École Nationale des Sciences de l'Informatique pour l'année universitaire 2013/2014. Nous présentons dans ce paragraphe l'organisme d'accueil ainsi que les secteurs d'activités dans lesquels il agit.

1.1.1 Fiche d'identité

Ce projet est réalisé au sein de la société TAC-TIC. Cette société crée en 2012, est un leader reconnu dans la fourniture de services de réseau IP/MPLS et de l'industrie des télécommunications en Tunisie.

1.1.2 Secteurs d'activités

TAC-TIC offre des services de réseau, de conseil aux entreprises, intégration de solutions open source et développement de logiciels spécifiques.

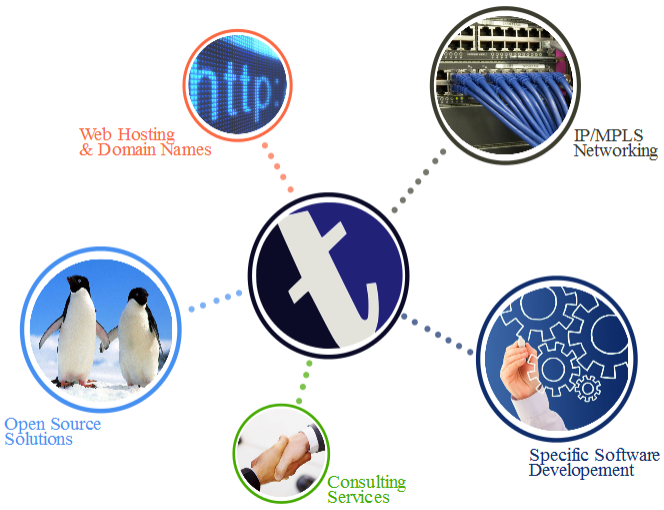


FIGURE 1.1 – Les activités de TAC-TIC

1.2 Présentation du sujet

Ce projet intitulé « Conception, réalisation et mise en place d'une application pour la gestion des projets » consiste en la conception et la réalisation d'une application web permettant d'offrir les principales fonctions d'une gestion efficace de projets. Elle permet de mettre en relation directe les clients avec les responsables de leurs projets.

Les employés, selon leurs rôles, gère leurs espace personnel (planification des évènements, remplissage de feuilles de temps, messages avec les autres membres de l'équipe, réalisation des taches, résolutions des tickets de maintenance,..).

Les clients gèrent aussi leurs espaces personnels. Ils auront la possibilité de suivre la réalisation de leurs projets en temps réel. Ils peuvent de même déclarer de nouveaux tickets ou valider celle déjà résolus comme ils peuvent contacter les responsables de leurs projets.

1.3 Choix méthodologique

Dans cette section nous présentons la méthodologie adoptée, le cycle de développement et le formalisme de conception.

1.3.1 Les méthodes agiles

Les méthodes agiles, peuvent être définies comme « des procédures de conception de logiciel qui se veulent plus pragmatiques que les méthodes traditionnelles. En impliquant au maximum le demandeur (client), ces méthodes permettent une grande réactivité à ses demandes, visent la satisfaction réelle du besoin du client, et non des termes du contrat de développement».

Afin de clarifier cette notion encore un peu vague, nous allons énoncer ses principes, ses fondements ainsi que les différences par rapport aux méthodes classiques :

- **Mettre en œuvre des individualités et des interactions**, plutôt que des procédés et des outils. Cela se manifeste par l'accent mis sur les êtres humains en tant qu'individus et sur l'expertise des équipes de développement, qui communiquent entre elles de façon très serrée et dans un esprit de confiance constant.
- **Produire un logiciel entièrement testé et qui fonctionne**, plutôt qu'une documentation claire. On rejoint ici les notions de développement itératif (notion de phases) et d'intégration continue (notion de builds journaliers), en insistant sur la simplicité et la robustesse du code produit (tests systématiques).
- **Collaborer avec le client**, plutôt que négocier un contrat. Le client devient un partenaire à part entière, qui participe au développement dans le sens où il détermine l'objectif à atteindre pour obtenir une réelle plus-value (qui seule justifie les efforts effectués pendant le développement).
- **Répondre aux modifications**, plutôt que suivre un plan. Il est bien évident que personne, pas même le client, ne peut appréhender avec précision l'ensemble des besoins dès le début du projet. Le développement agile vise à atteindre un compromis entre les spécifications initiales présentées aux développeurs (et sur lequel se fonde le planning) et le résultat final qui bien souvent s'en éloigne un peu voire beaucoup, en absorbant les modifications tout au long du cycle de développement. Cela réclame des outils de suivi et une attitude constante de coopération avec le client.

1.3.2 Un comparatif des méthodes agiles

Au cours de cet paragraphe nous allons essayer de dégager les différences entre quelques unes des méthodes de développement agiles les plus connues, et ceci en fournissant un tableau comparatif de ces dernières.

Le tableau suivant, Tab 1.1, compare sommairement les méthodes courantes suivantes :

- Extreme Programming (XP).
- Rational Unified Process (RUP).
- Feature-Driven Development (FDD).
- Scrum

Méthode	**Points clés**	**Inconvénients**
XP	– Développement guidé par les besoins du client. – Equipe réduite, centrées sur les développeurs par binomes. – Builds journaliers, amélioration constante, adaptativité aux modifications.	– Focalisation sur l'aspect individuel du développement, au détriment d'une vue globale et des pratiques de management ou de formalisation. – Risque de manque de contrôle et de structuration en laissant les développeurs trop libres de dériver par rapport aux fonctions de l'application.
RUP	– Processus complet assisté par des outils. – Rôles bien définis	– Lourd, largement étendu, il peut être difficile à mettre en œuvre de façon spécifique. – Convient pour les grands projets qui génèrent beaucoup de documentation.
Scrum	Petites équipe, itération de 30 jours, réunions journalières.	La mise en œuvre du développement n'est pas précisée, seule compte la gestion des ressources humaines.
FDD	– Procédé bien défini et simple, orienté objet et basé sur le développement. – Itératios très coutres	Uniquement centré sur le développement.

TABLE 1.1 – Tableau comparatif de quelques méthodes agiles

1.3.3 La méthodologie Scrum

Nous avons choisi la méthodologie Scrum pour la conception et le développement de notre application.

1.3.3.1 Introduction à la méthodologie Scrum

Le terme Scrum(qui signifie « mêlée »en rugby) se rapproche plus d'une gestion de ressources humaines plutôt que d'une réelle méthode de développement. Il s'agit ici de ne pas oublier le côté humain du développement.

Les principales caractéristiques de Scrum sont :

- Identifier les changements très tôt.
- Donner toute confiance aux développeurs et les laisser faire leur travail.
- Faire des itérations variantes (généralement de 30 jours), appelés aussi « sprints »pour laisser le temps de coder. Chaque itération a un objectif bien précis ou « backlog » et fournit une nouvelle fonctionnalité testée (une démonstration est faite à la fin de chaque sprint).
- Faire des réunions tous les jours (daily meeting) et chaque semaine (weekly meeting) pour encadrer les équipes et recaler les objectifs.

1.3.3.2 Le choix de la méthodologie Scrum

Davantage qu'une méthode formelle, Scrum, illustré par la figure 1.1, peut être vu comme un framework méthodologique dont l'implémentation doit être ajustée en fonction des caractéristiques techniques, organisationnelles et culturelles des projets qui souhaitent la mettre en oeuvre (Scrum, au demeurant, ne limite pas son champ d'application aux seuls projets informatiques : ses principes sont applicables pour toute activité visant à produire un résultat).

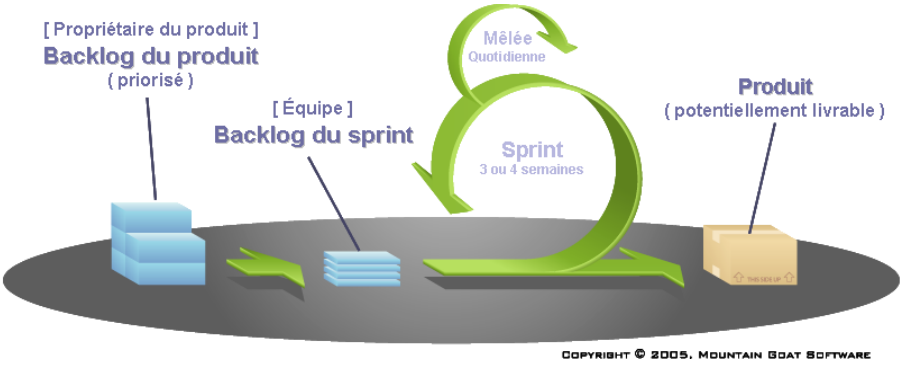


FIGURE 1.2 – Schéma illustratif de SCRUM

Dans ses grandes lignes, Scrum définit un jeu minimal d'acteurs, de cérémonies et d'artefacts qui permettent de relever les défis principaux du développement incrémental : la planification, la gestion du temps et la gestion des obstacles. Scrum est entièrement piloté par la Valeur Métier.

La gestion des risques, en particulier, est réalisée au travers de ce prisme. Scrum identifie trois acteurs :

- **Le Product Owner** ((Directeur de Produit), qui possède l'expertise fonctionnelle et est à même de réaliser les arbitrages nécessaires à la priorisation des développements. Son rôle est absolument essentiel, et son respect des règles du jeu est la pierre angulaire du succès d'un projet agile.
- **membre de l'Equipe**, et dont la tâche principale est d'optimiser la capacité de production de l'Equipe en l'aidant à travailler de façon autonome et à s'améliorer constamment. Il est également le garant de la bonne implémentation de Scrum.
- **l'Equipe, dont la taille doit être réduite**, et qui prend en charge le développement du produit (planification, conception, codage, tests, documentation) sans spécialisation des rôles. La particularité d'une Equipe Scrum est d'être «auto-organisée », et donc dépourvue de hiérarchie. Cet aspect constitue une rupture radicale avec les approches managériales traditionnelles, qui privilégient un contrôle centralisé généralement incarné par le Chef de Projet.

Les avantages cités ci-dessus se révèlent particulièrement, bien adaptés aux projets de fin d'étude dont les objectifs et la limitation temporelle sont parfaitement délimités et connus. La pratique de la méthode Scrum nous a donné l'opportunité d'être intégré au sein de ce processus et de participer aux cycles de développement. Après avoir fait le choix de la méthodologie,

qui est une étape primordiale dans le cycle de développement d'un produit informatique, nous exposerons dans le paragraphe suivant la problématique auquel nous sommes amenés à développer une solution.

1.3.4 Formalisme adoptée

UML (Unified Modeling Language) est un langage de modélisation graphique orienté objet de troisième génération à base de pictogrammes. Dans le cadre de notre spécification, le choix UML a été effectué à cause de la possibilité de modification et la réutilisabilité et la modularité qui sont les qualités reconnues de cette approche.

UML fournit un moyen astucieux permettant de représenter diverses projections d'une même représentation grâce à ces différents diagrammes. En effet, il couvre l'aspect statique et dynamique d'un système selon ses différents diagrammes. Il définit pour cela dix diagrammes qui sont subdivisés en des vues statiques (qui représentent « physiquement » le système à modéliser au moyen de diagrammes d'objets, de classes, de paquetage, de cas d'utilisation, de composants, de déploiement et enfin d'architecture) et des vues dynamiques (qui traduisent le fonctionnement du système au moyen de diagrammes de séquence, de collaboration, d'états transitions et d'activités).

Pour éviter de surcharger le rapport et d'entrer dans certains détails techniques, nous ne présenterons que quelques diagrammes que nous avons jugés utiles pour comprendre le projet à savoir :

- Le diagramme des cas d'utilisation : Il permet de structurer les besoins des utilisateurs et les objectifs correspondants de notre système en identifiant ses utilisateurs et leurs interactions.
- Le diagramme de séquence : Il permet une représentation temporelle des objets et de leurs interactions.
- Le diagramme de paquetage : Il permet de modéliser l'application sous forme de paquetages. Ceci offre un niveau d'abstraction élevé et permet de présenter la modularité de l'application.
- Le diagramme de classes : Il permet de présenter les classes et les interfaces de notre système ainsi que les différentes relations entre celles-ci

Conclusion

Ce chapitre a donné l'occasion de présenter dans un premier temps la société accueillante TAC TIC puis le cadre du sujet, ainsi que les objectifs que notre travail vise à atteindre. En vue de suivre un avancement logique dans ce rapport, une étude théorique concernant l'état de l'art fera l'objet du prochain chapitre.

Chapitre 2

Etat de l'art et étude de l'existant

Introduction

Dans ce chapitre, nous commençons une étude sur le système de gestion de projet existant adopté par TAC TIC et ses limites . Puis, nous mettons l'accent sur quelques solutions concurrentielles existantes sur le marché qui peuvent répondre aux besoins de TAC TIC

2.1 Situation Actuelle

Afin d'assurer une bonne gestion de ses projets, de temps et de ressources La société TAC-TIC utilise Dotproject comme logiciel de gestion de projet et Ehour pour la gestion des feuilles de temps.

2.1.1 Dotproject

DotProject est l'un des outils de gestion les plus populaires Open Source du projet. Il permet de créer, suivre et maintenir les projets. Il fournit des outils de gestion de projets d'entreprise, qui comprennent un gestionnaire de contacts, un système de notification par courrier électronique et une application en ligne pour créer et gérer des projets. Des codes de couleur intuitifs pour indiquersi le projet est « dans le rouge ».

L'outil est développé en PHP et s'interface nativement avec une base de données MySQL pour le stockage des données de projets. Il est libre, sous licence General Public License (GPL) de GNU et BSD, ce qui signifie qu'il s'agit d'un logiciel libre, fourni en l'état, sans aucune garantie.

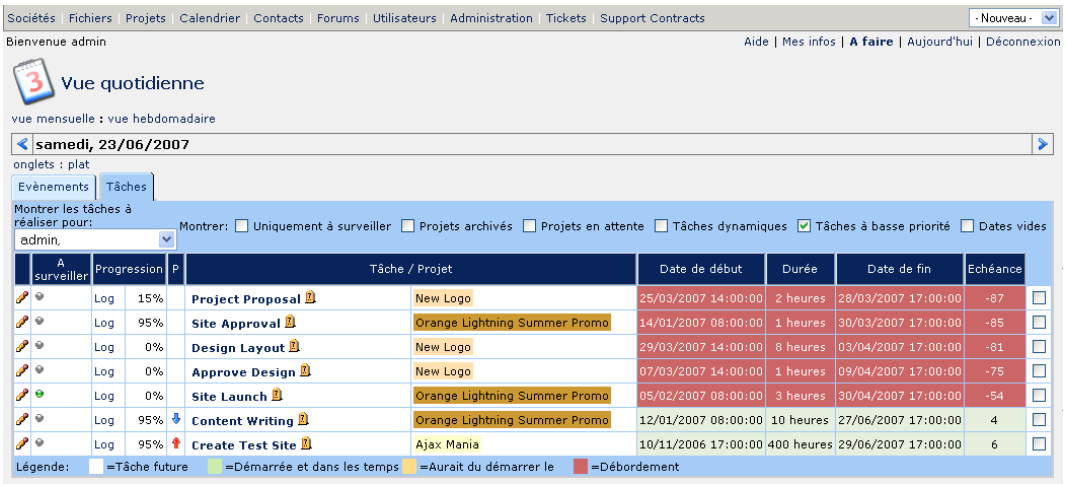


FIGURE 2.1 – DotProject : Tableau de bord d'un utilisateur

Les fonctionnalités principales de DotProject sont :

- Gestion des utilisateurs et permissions.
- Gestion de bugs liés au projet et notification par mail.
- Interface gestion Client/Compagnie.
- Liste des projets et affichage hiérarchique des taches, des sous tâches, et visualisation graphique du projet via des diagrammes de Gantt (Utiles pour les chefs de projet).
- répertoire de contact partagé.
- Calendrier privé et partagé.
- Forums de discussions liés à un projet.

2.1.2 Ehour

La valeur du temps est reconnue par tous ceux qui visent à assurer l'efficacité et la gestion efficace des ressources. Les personnels peuvent suivre l'utilisation de leur temps contre les activités afin de maximiser leur productivité, tandis que les organisations comptent sur les feuilles de temps pour garder une trace de la répartition du travail, statut et d'autres indicateurs de gestion pour maximiser la production dans le minimum de temps possible.

En gardant ces objectifs en considération, et l'importance d'une interface conviviale et des infrastructures de documentation complets aider les gestionnaires à tirer la meilleur partie de leur temps. EHour est un utilitaire web de gestion de feuilles de temps précis qui vous informe sur le temps consacré à diverses tâches dans de multiples projets. Il prend en charge une va-

riété de langues dont l'anglais, néerlandais, français, italien, allemand et polonais. Avec des caractéristiques comme Excel Report génération, des rôles bien défini et une localisation des options de calcule de couts, cette application promet d'ajouter de la valeur à l'environnement de travail.

Les fonctionnalités principales de Ehour sont :

- créer plusieurs projets par client, attribuer à plusieurs utilisateurs d'un même projet.
- créer des projets par défaut à tous les utilisateurs qui doivent être attribués (jours de congé, les congés de maladie, etc) afin que vous puissiez garder une trace de ces heures ainsi des rapports détaillés.
- localisation configurable et monnaie. eHour est disponible en anglais, néerlandais, français, italien, allemand (partielle) et polonais (partielle).
- séparation de l'utilisateur, gestionnaire de projet, reporting et l'administration rôle.
- possibilité de verrouiller les feuilles de temps d'exportation vers Excel de tous vos rapports.
- interface utilisateur basé sur le web, sans logiciel doit être installé e sur les ordinateurs de bureau.

Ehour est développé en Java en utilisant Spring, Hibernate et technologie Wicket ce qui assure sa portabilité.

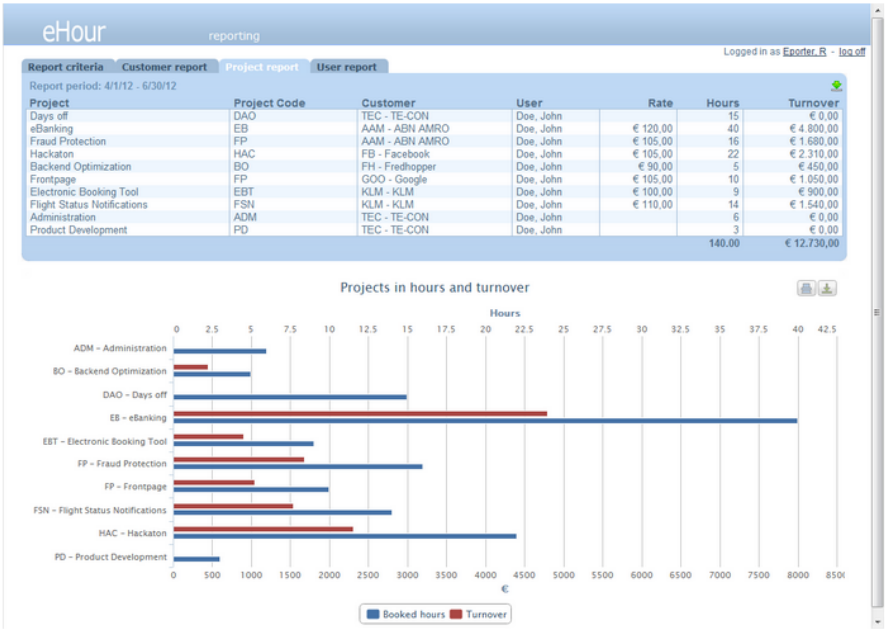


FIGURE 2.2 – Ehour : Génération des rapports et des statistiques

2.1.3 Les Limites des solutions existantes

DotProject offre d'excellentes fonctionnalités pour communiquer avec vos contacts et ressources. Néanmoins, il n'a pas été conçu comme outil de travail collaboratif (en anglais, groupware). Le terme « travail collaboratif » est généralement utilisé pour décrire les applications locales ou web qui permettent à des utilisateurs de travailler en collaboration. Elle comprend souvent des forums, des outils de discussion en ligne, des wikis et des messageries électroniques qui sont généralistes par nature.

Les limites de DotProject :

- Impossible de restreindre totalement l'accès des utilisateurs à d'autres projets que les leurs.
- Impossible de regrouper les tâches en «lots ».
- Impossible d'éviter quelques petits bugs qui demeurent (problèmes de dates).

Les limites de Ehour :

- L'absence de notion de taches ce se qui implique le manque de détails l'ord de la consultation des rapports
- Le remplissage quotidien des feuilles de temps pour chaque projet est unique ce qui rond obliger le regroupement de tous les taches dans une seule fois

2.2 Les Solutions disponibles

La recherche logicielle s'est concentrée sur Internet, par le biais de forums ayant déjà évoqué le sujet, du moteur de recherche Google, et du site Wikipédia qui propose une liste de logiciels de gestion de projet, libres ou propriétaires.

Une fois la liste de logiciels établie, les sites internet de chacun ont été consultés pour avoir les informations souhaitées. Les démonstrations en ligne ont également permis de faire des tests.

2.2.1 Les critères de recherche

Pour qu'une solution soit retenue, elle doit vérifier les critères suivants :

- Ouverture et possibilité de mise à jour (facile à comprendre et à modifier pour l'intégration de nouvelles fonctionnalités ou mise à jour d'autres).
- Plate-forme remplissant le maximum de tâches.

– Facilement compréhensible.

2.2.2 Quelques plateformes étudiées

Dans cette partie, nous allons présenter les systèmes pouvant répondre aux exigences de TAC TIC, les plus importants et les plus satisfaisants aux critères de recherche.

2.2.2.1 ProjeQtOr

ProjeQtOr est un logiciel de gestion de projets open source, qui regroupe dans un outil unique toutes les fonctionnalités nécessaires à l'organisation d'un projet. Il reste simple, facile à utiliser au quotidien, tout en couvrant un maximum de fonctionnalités de la gestion de projet :

– gestion multi-projets collaborative, multilingue, ergonomique simple à utiliser.
– gestion des activités planifiées, des tickets, des actions, des dépendances entre activités / jalons / projets.
– gestion des charges, des imputations, du planning (Gantt), du portefeuille de projets.
– gestion des coûts des ressources, des dépenses, des commandes, de la facturation.
– gestion des risques, des opportunités, des problèmes.
– gestion des réunions, des réunions périodiques, des questions, des décisions.
– gestion de la documentation et des versions de documents, avec processus simple d'approbation.
– gestion des exigences, des cas de tests et des sessions de tests
– gestion de workflows
– envoi de mails, génération d'alertes sur indicateurs

Il est nécessaire de paramétrer celui-ci avant de commencer. Notons que c'est une tache qui prend du temps pour ce logiciel car il est très complet.

2.2.2.2 GanttProject

Ce logiciel est relativement connu. Il propose une interface simple et intuitive pour la création de Gantt. L'avantage de GanttProject est qu'il n'y a pas besoin de configuration. Ceux qui n'ont pas besoin de nombreux outils, mais simplement d'un moyen de planification et de visualisation, seront réellement intéressés par ce logiciel.

En contre partie, il n'y a pas de gestion multi-projet et la durée minimum d'une tache est d'un jour. Le plus ardu est d'affecter une ressource, car pour cela il faut aller dans les propriétés

de la tache et lui donner la ou les ressources voulues.

Une remarque sur ce logiciel est qu'il est la référence pour le format Gantt, et beaucoup de logiciels permettent l'importation de ses projets.

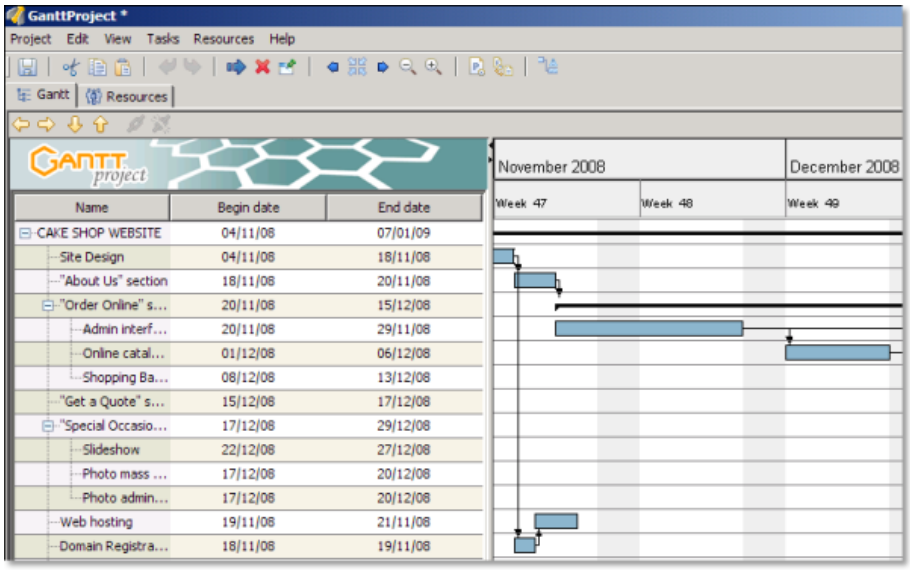


FIGURE 2.3 – Apperçu de GanttProject

2.2.2.3 Redmine

Redmine est un logiciel de gestion de projet relativement performant, qui tire avantage de sa communauté et de ses plugins pour renforcer ses fonctions et s'adapter aux besoins de chaque utilisation. Son interface sobre offre une certaine clarté malgré la quantité d'informations qui peut être gérée. Le pas le plus dur à franchir avec ce logiciel se situe lors de sa prise en main, car sa logique n'est pas toujours intuitive. En plus, son Gantt est peu évolué (un outil de suivi et non pas de planification).

2.2.3 Autre solutions

La documentation sur les solutions commercialisé n'est pas assez développé mais certaine d'entre eux offrent Les démonstrations en ligne. Parmi les solutions répondant le mieux au besoins de TAC-TIC, on peut citer :

- www.teamwork.com
- www.twproject.com

2.3 Solution Envisagé

Notre application met en contact direct tous les intervenants dans la réalisation d'un projet. Elle offre aux employés, selon leurs profils, un accès vers les données des projets auxquels ils participent, un calendrier personnel avec gestion des réunions et le suivie de leurs feuilles de temps par génération des rapports et des statistiques. L'application sera occupé aussi d'un système de messagerie et d' un système de notification par email.

D'autre part l'application à réaliser interagit avec les clients en leurs offrants une interface paramétrable leur permettant le suivi de leurs projets dès les premières études jusqu'à l'archivage. Avec un système de support, l'application assure la déclaration et la validation des bugs, pannes et assurant une maintenance des produits livrés.

Finalement, un outil d'administration qui possède les droits illimités de gestion du site et des différents utilisateurs (client ou employés).

Conclusion

Ce chapitre nous a permis de préciser le cadre dans lequel se situe ce projet, ainsi que de présenter ses objectifs visés. En plus, nous avons eu l'occasion d'étudier des concepts qui seront utilisés dans le cadre du projet. Enfin, nous avons présenté une étude de l'existant pour pouvoir cibler les points importants de l'application. Dans le chapitre suivant, nous allons spécifier les besoins fonctionnels et non fonctionnels de notre application. Nous détaillerons ensuite quelques cas d'utilisation que nous avons pu dégagés.

Chapitre 3

Spécification et analyse des besoins

Introduction

Dans tout système, les fonctionnalités doivent être mises en relation avec un ensemble de besoins utilisateurs. Ces besoins définissent les fonctions que les utilisateurs s'attendent à voir par le système. La spécification des besoins forme les fondations sur lesquelles l'architecture du système est construite. Tout au long de ce chapitre nous allons définir les besoins, identifier les acteurs et les activités desquels nous déduisons assez facilement les cas d'utilisations de notre application.

3.1 Étude des besoins

L'analyse du sujet nous a permis de cerner les fonctionnalités du système que doivent être à la disposition de l'utilisateur. Les besoins ainsi dégagés sont classés en fonctionnels et non fonctionnels.

3.1.1 Les acteurs du système

Les acteurs qui manipuleront notre application sont :

- **Le client** : c'est le propriétaire des projets. Il possède un abonnement a partir duquel il peut suivre l'évolution de la réalisation des projets ou déclarer les tickets.
- **Les employés** : ce sont les employés de société. Il participe à la réalisation des taches, rependent aux tickets et participes au discutions. Les droits attribuées aux employées sont spécifiés selon leur profiles. Pour la présentation de l'ensemble des fonctionnalités

de l'application, notre application dispose de deux types d'employés par défaut : Le chef de projet et le développeur.

– **L'administrateur du site** : il possède tous les droits. Il gère surtout les profils décrivant les droits d'accès. En effets, Les droits attribuées aux utilisateurs (clients ou employés) sont spécifiés selon des profiles. L'administrateur aura la possibilité de manipuler les profils en leur ajoutant ou supprimant des droits ou même créant des nouveaux.

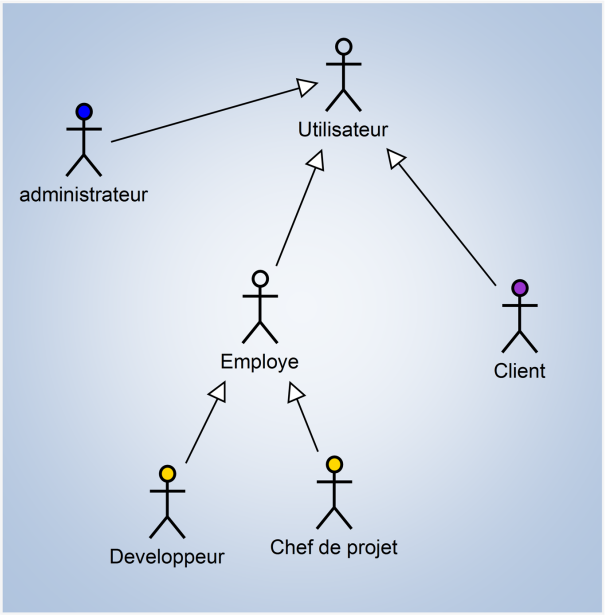


FIGURE 3.1 – Les différents utilisateurs de l'application

Le schéma ci dessus présente l'hiérarchie des différents utilisateurs de l'application.Nous présenterons dans la suite les principaux cas d'utilisation qui assurent toutes les tâches exécutées par le système.

3.1.2 Besoins fonctionnels

Les besoins fonctionnels doivent répondre aux exigences du système en termes de fonctionnalités. Ils constituent une sorte de contrat par rapport au comportement du système .

3.1.2.1 Les besoins fonctionnels pour un administrateur

Après l'authentification, l'administrateur peut :

– Gérer les profils et les droits.

– Gérer les utilisateurs.

- Gérer les projets.
- Préciser les informations concernant la société.
- Gérer des clients.
- Consulter les statistique et les rapports.
- Gérer son propre calendrier et celle des événements.
- Suivre les actualités à travers les notifications.
- Remplir ses feuilles de temps.

3.1.2.2 Les besoins fonctionnels pour les employés

a. Chef de projet

- Gérer ses projets.
- Gérer son équipe.
- Communiquer avec les clients.
- Consulter les statistiqueet les rapports concernant ses projets.
- Gérer son propre calendrier et celle des événements.
- Recevoir des notifications sur les actualités.
- Remplir ses feuilles de temps.
- Suivre l'avancement des employés dans leurs taches.

b. Développeurs

- Participer au discussions des projets auxquels il fait partie.
- Gérer son propre calendrier.
- Partager des fichiers.
- suivre les actualités à travers les notifications.
- Remplir ses feuilles de temps.

3.1.2.3 Les besoins fonctionnels pour les clients

- Le système permet au client authentifié de déclarer des tickets.
- L'utilisateur authentifié peut discuter avec le responsable de son projet.

3.1.3 Besoins non fonctionnels

Dans l'intention de réussir notre tâche, l'application doit vérifier quelques propriétés et doit tenir compte de certaines contraintes et exigences.

a. Convivialité

Plusieurs acteurs sont susceptibles interagir avec le système, pour cela, un niveau de convivialité raisonnable devrait être respecté. Ceci en utilisant des procédures de navigation entre les contextes d'utilisation qui soient évidents et facilement accessibles pour la plupart des acteurs.

b. Interface Riche

L'interface de la plateforme doit être dotée de toutes les caractéristiques d'une interface client riche ; gestion de la synchronisation entre les vues, gestion des menus, gestion des permissions.

c. Champs obligatoires

Si le champ est obligatoire, il faut qu'il soit précédé d'un flag au niveau du label du champ, ex : *. Si un champ obligatoire n'est pas renseigné, il faut prévenir l'utilisateur et positionner un indicateur sur le ou les champs qu'il doit absolument remplir.

d. Exigences de portabilité

Le produit doit être développé en PHP, Le produit doit être fonctionnel sur la majorité des navigateurs web.

e. Sécurité

Seuls les administrateurs pourront consulter les informations personnelles des membres, Les droits d'accès doivent être bien respectés, Le serveur de l'application doit être protégé contre les menaces du web tel que le « SQL injection ».

3.2 Les diagrammes de cas d'utilisations

Cette phase a pour objectif de décrire le comportement attendu de l'application. Pour cela nous nous basons sur le diagramme des cas d'utilisation qui représente un élément essentiel de

la modélisation orientée objet. Elle permet de modéliser les fonctionnalités de l'application de point de vue besoin utilisateur. Elle sert aussi à définir le produit à développer, et le modéliser.

3.2.1 Le cas d'utilisation général

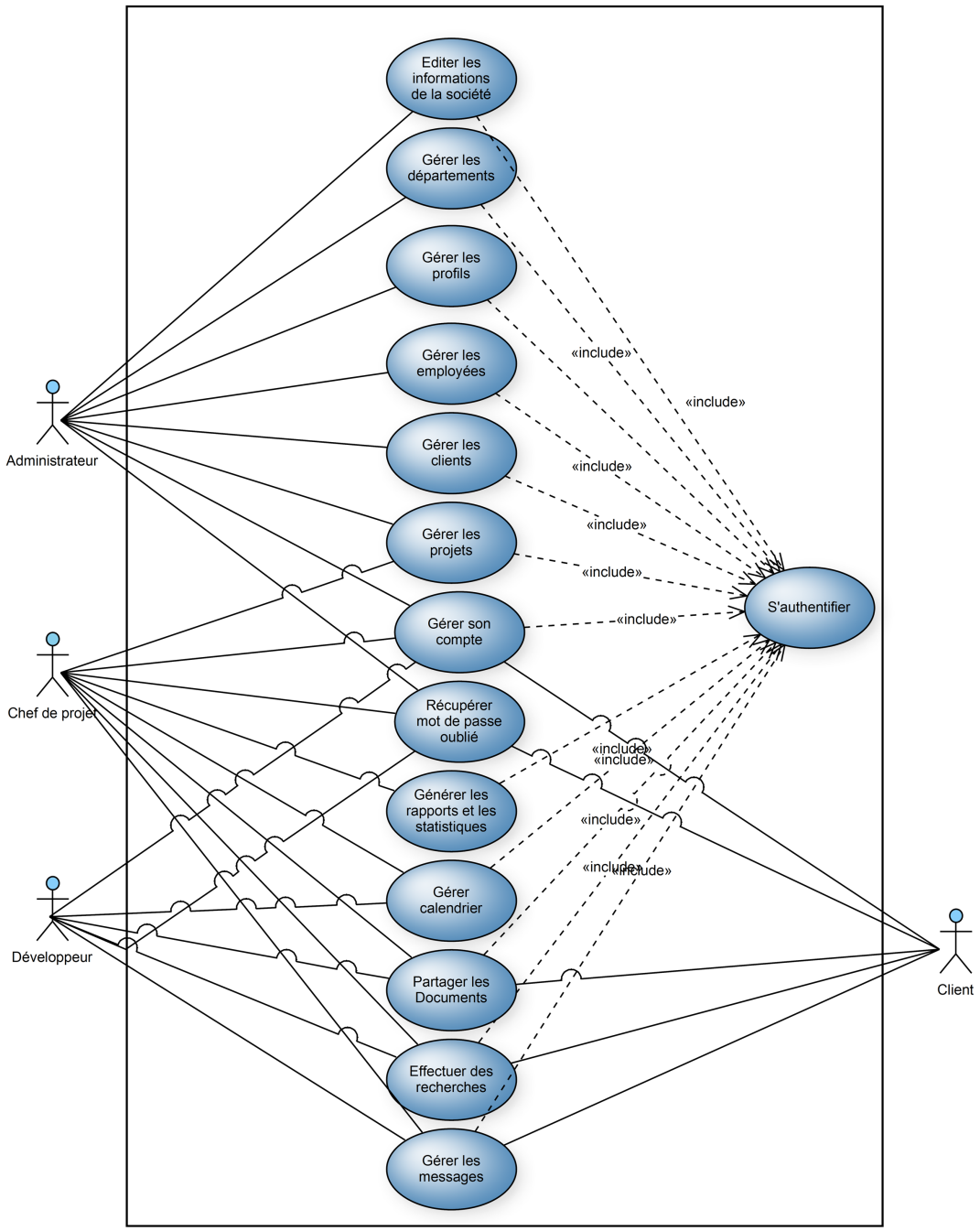


FIGURE 3.2 – Diagramme de cas d'utilisation général

La figure ci-dessus présente le diagramme des cas d'utilisation général. Nous voyons le différents acteurs du système(client, chef de projet, employé et administrateur).Nous détaillerons dans la suite pour chaque utilisateur les cas d'utilisation accessibles pour lui.

3.2.2 Les cas d'utilisation de l'administrateur

La figure ci dessous présente le rôle de l'administrateur de l'application. En effet, il est capable de :

- Gérer son compte
- Gérer les profils
- Gérer les employés.
- Gérer les projets.
- Gérer les informations de la société.
- Gérer les clients.
- Récupérer son mot de passe

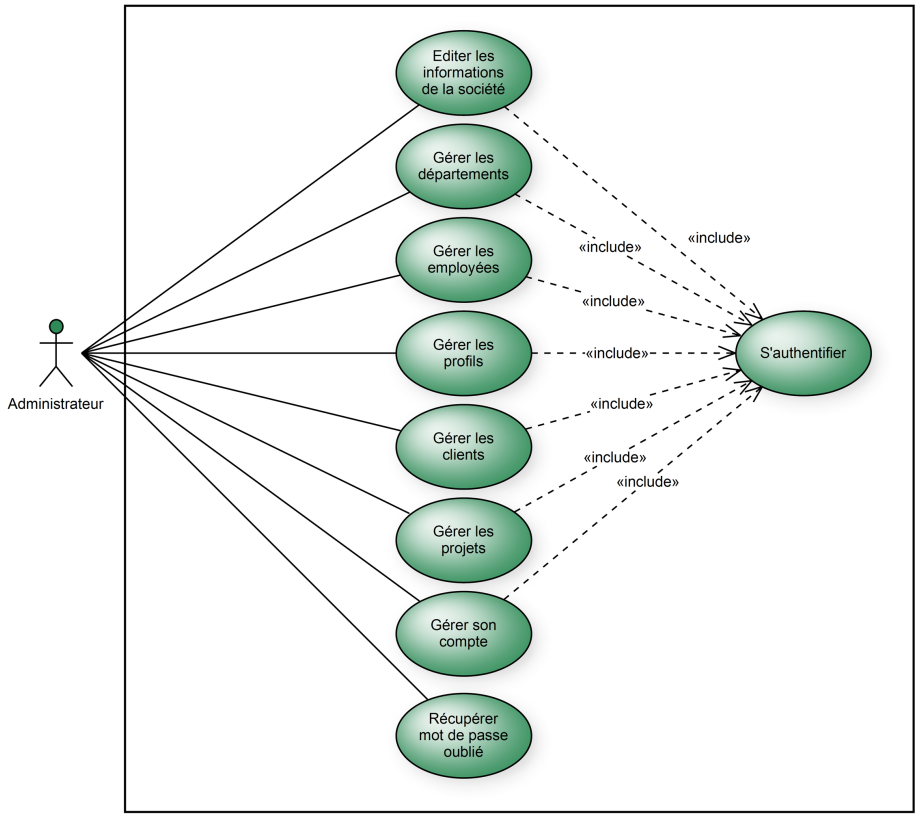


FIGURE 3.3 – Diagramme de cas d'utilisation général :Administrateur

Nous détaillons le cas d'utilisation principale de l'administrateur : « Gérer profiles ». Dans ce cas l'administrateur peut :

- Ajouter un profile
- Modifier un profile
- Consulter la liste des profiles
- Supprimer un profil

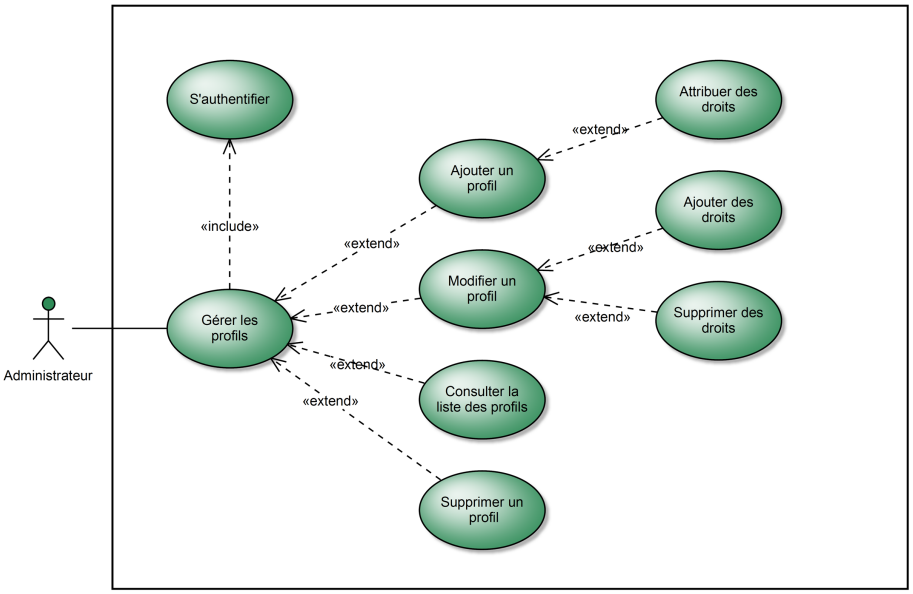


FIGURE 3.4 – Diagramme de cas d'utilisation : administrateur/Gérer profils

3.2.3 Les cas d'utilisation du chef de projet

Le chef de projet, principalement, peut en accédant à son compte :

- Gérer les projets
- Effectuer des recherches
- Gérer les messages
- Partager les documents
- Générer les rapports et les statistiques
- Gérer son calendrier
- Gérer son compte
- Récupérer le mot de passe oublié

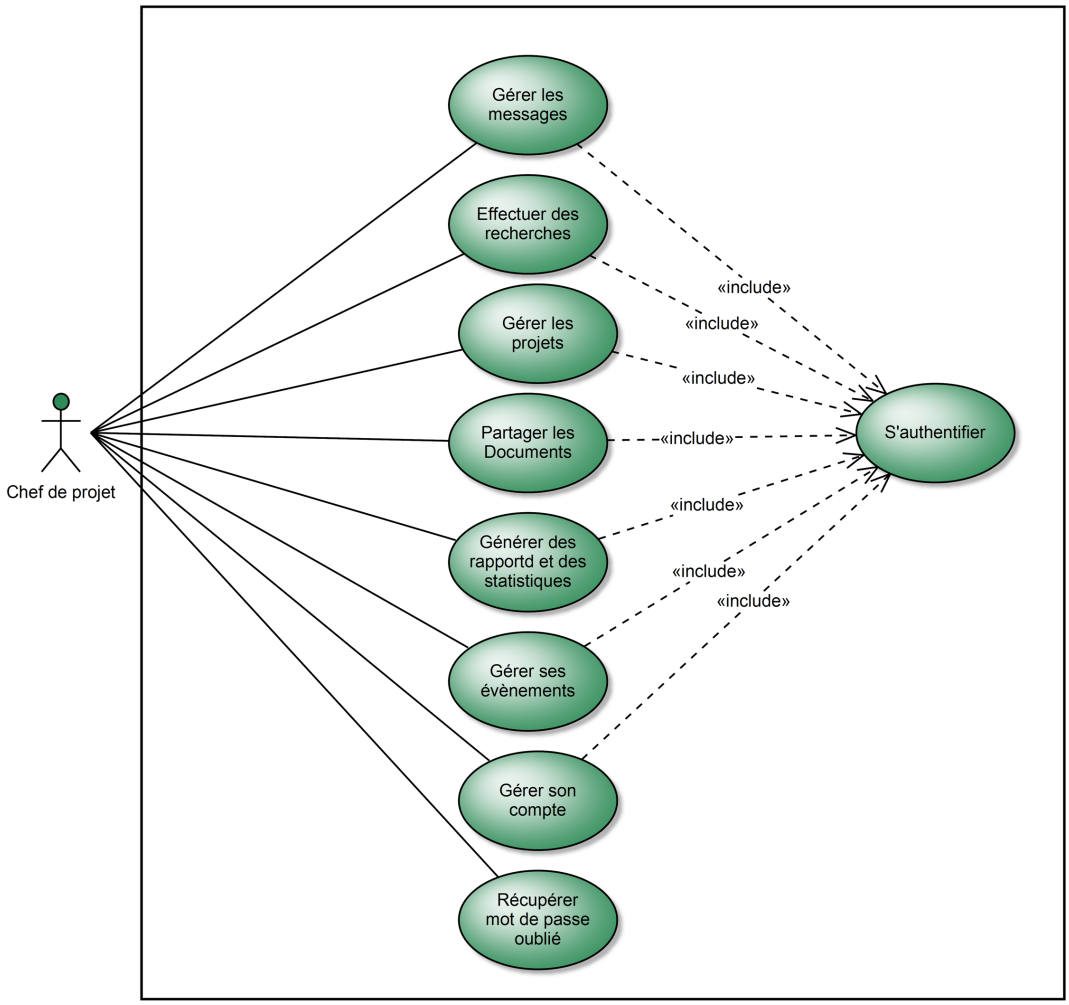


FIGURE 3.5 – Diagramme de cas d'utilisation général : Chef de projet

Nous détaillons par la suite chaque cas d'utilisation qui nécessite une clarification.

Pour le cas d'utilisation « Gérer les projets » le chef de projet est capable de :

- Gérer les taches.
- Suivre avancement.
- Gérer les tickets.
- Gérer le diagramme de Gantt.
- Archiver un projet.
- Gérer les types des projets.

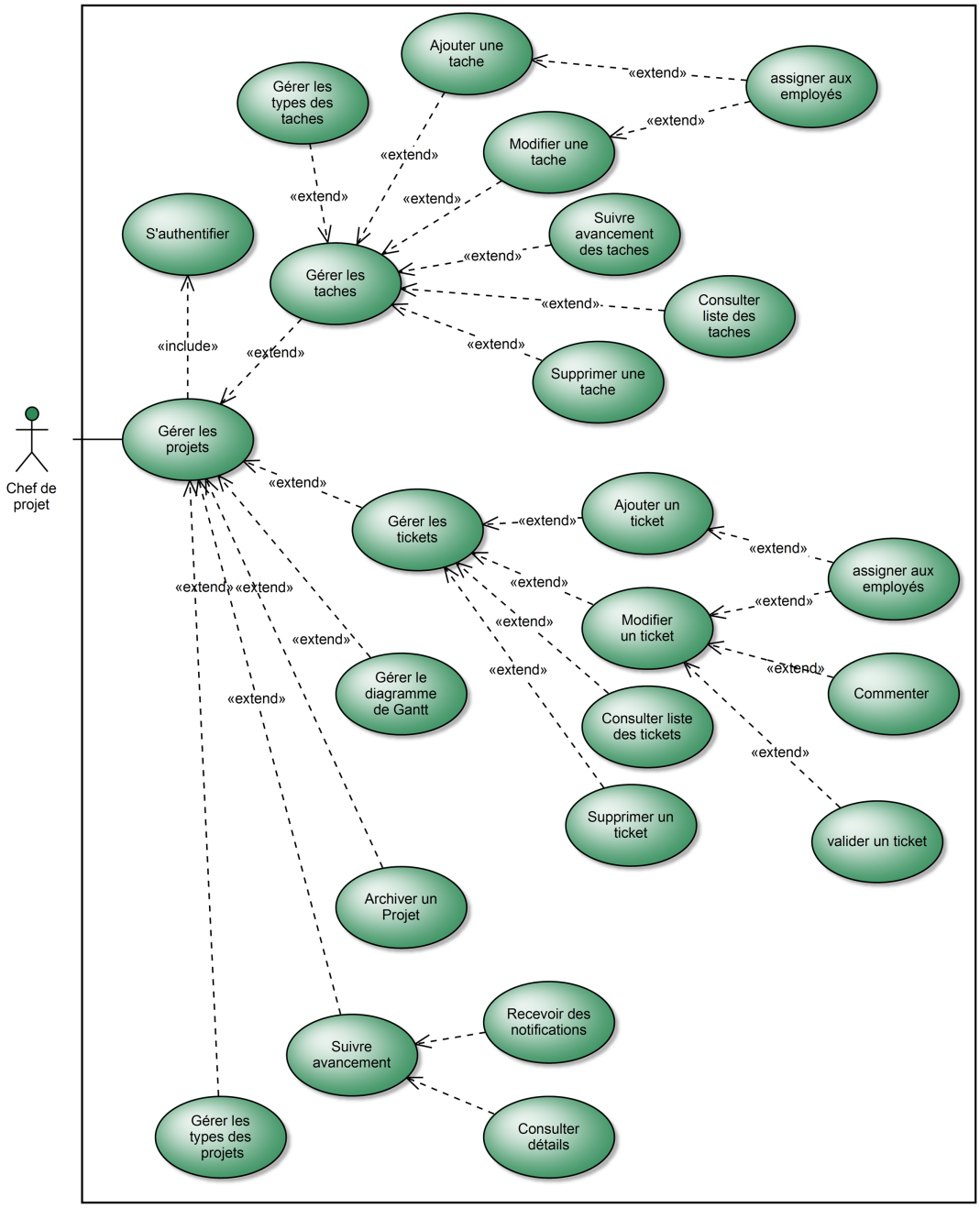


FIGURE 3.6 – Diagramme de cas d'utilisation : chef de projet/Gérer projets

le chef de projet aura la possibilité de gérer sa boite de messagerie à travers : *

- Envoyer un message.
- Gérer les réponses (les messages reçus).
- Supprimer un message.

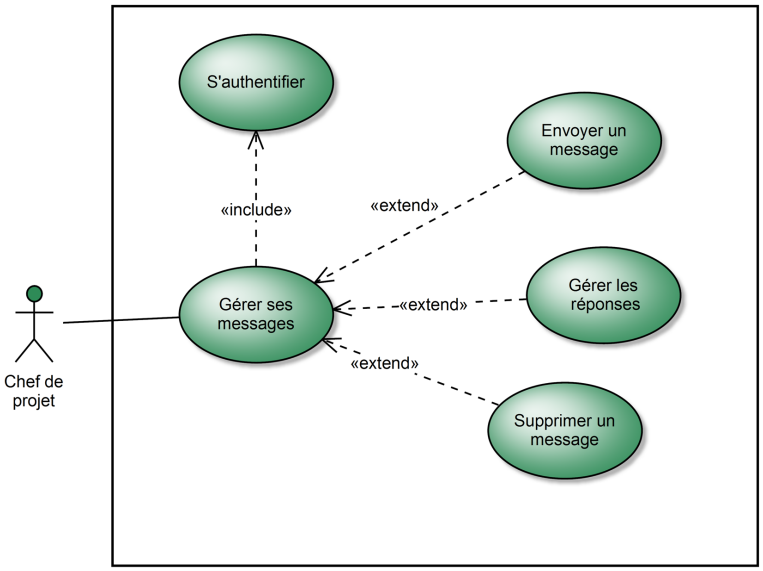


FIGURE 3.7 – Diagramme de cas d'utilisation : chef de projet/Gerer messages

3.2.4 Les cas d'utilisation du développeur

Le chef de projet, principalement, peut en accédant à son compte :

- Participer au discussions des projets auxquels il fait partie.
- Gèrer son compte.
- Gèrer ses projets.
- Gèrer son calendrier.
- Partager des fichiers.
- Remplir ses feuilles de temps.
- Effectuer des recherches
- Récupérer le mot de passe oublié.

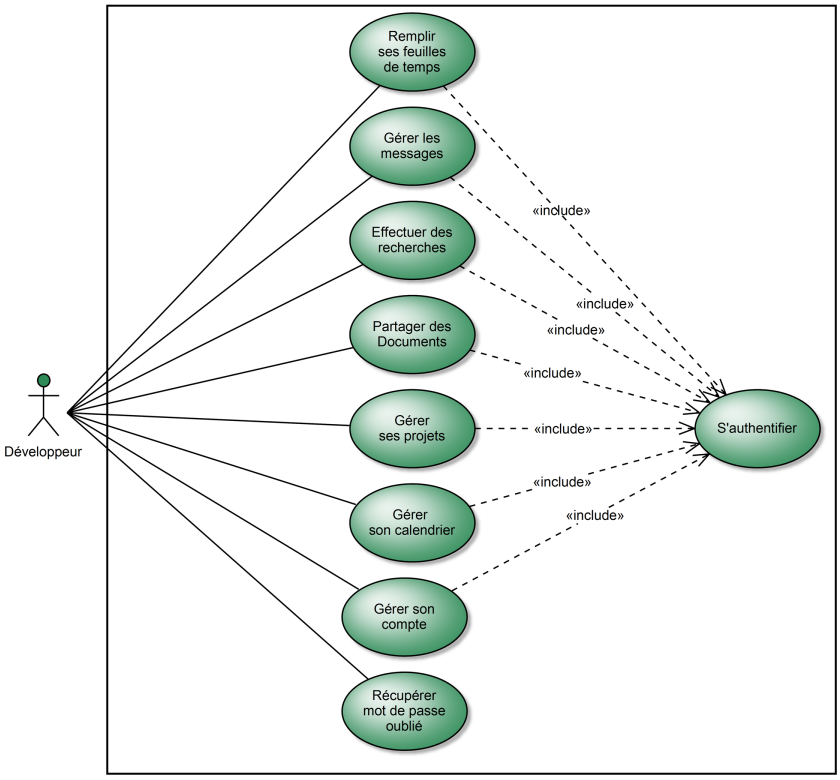


FIGURE 3.8 – Diagramme de cas d'utilisation général : Développeur

Pour le cas d'utilisation « Gérer Calendrier», le developpeur est appelé à :

- Ajouter un évènement.
- Modifier un évènement.
- Supprimer un évènement.
- Organiser des réunions (inviter des utilisateurs).

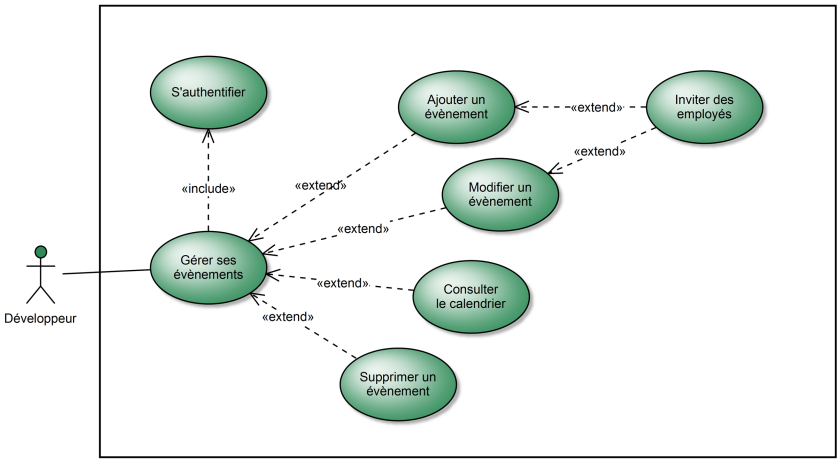


FIGURE 3.9 – Diagramme de cas d'utilisation : Développeur/Gerer calendrier

3.2.5 Les cas d'utilisation du client

Le client présente une entreprise qui est abonnée dans le site et qui possède généralement un ensemble de projet prédéfini. Les fonctionnalités d'un client alors sont :

- Gérer son compte.
- Gérer ses messages.
- Consulter ses projets.
- Partager des documents.
- Déclarer des tickets.
- Effectuer des recherches.
- Gérer des tickets.

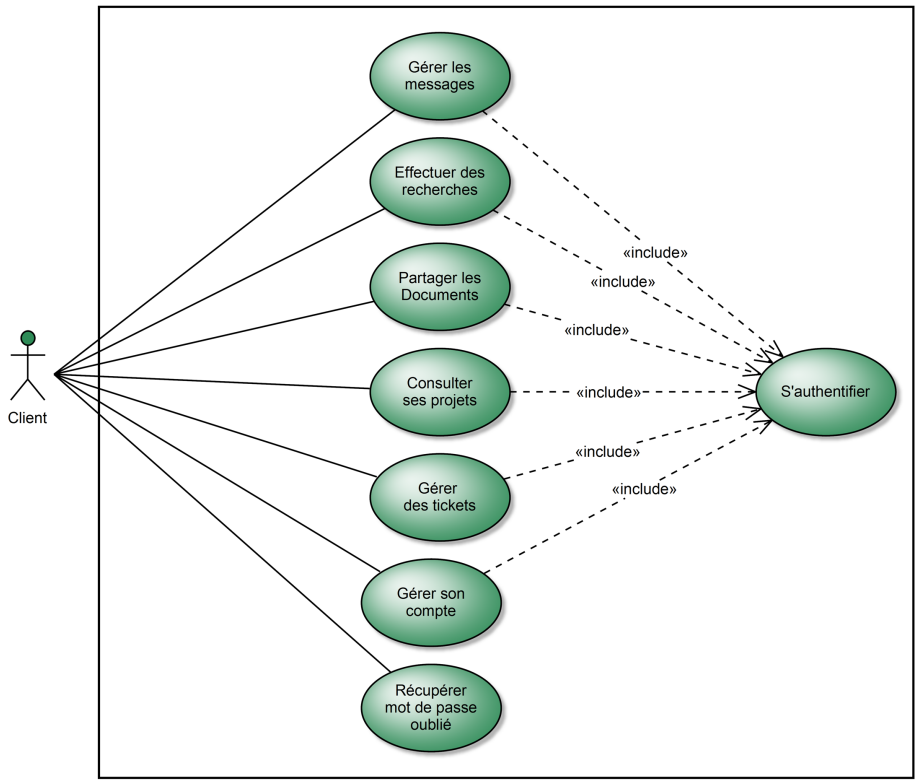


FIGURE 3.10 – Diagramme de cas d'utilisation général : Client

Notons que le client ne s'inscrit pas automatiquement au site tmais suite à une demande à l'administrateur du site qui a le droit d'autoriser à une entreprise de coopérer avec lui.

Pour le cas d'utilisation « Gérer des tickets », le Client est capable de :

- Déclarer des tickets.
- Commenter un ticket

– Valider un ticket.

– Supprimer un ticket.

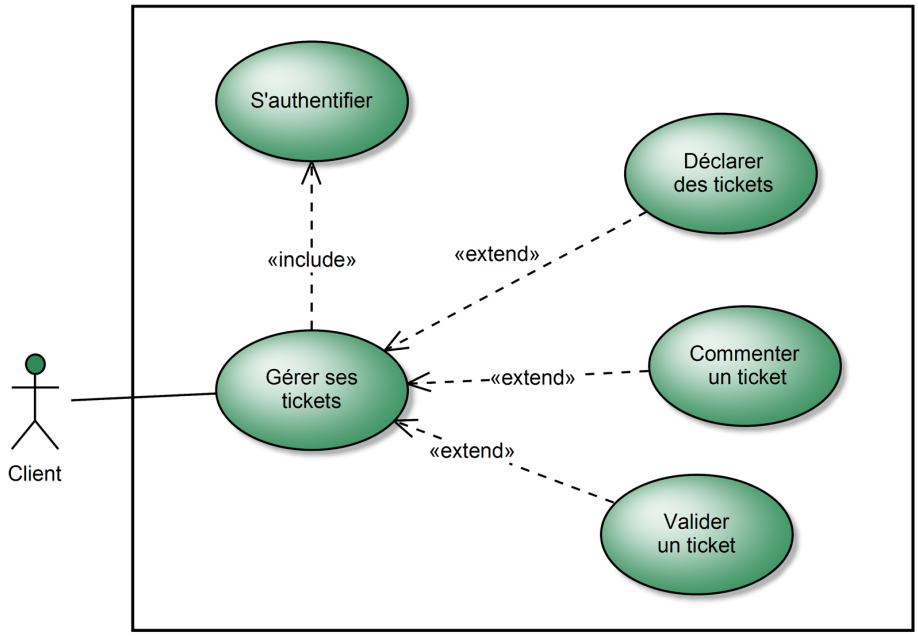


FIGURE 3.11 – Diagramme de cas d'utilisation : Client/Gerer tickets

3.3 Diagrammes de séquence

Avec les diagrammes de séquences, l'UML fournit un moyen graphique pour représenter les interactions entre les objets à travers le temps [N1]. Ces diagrammes montrent typiquement un acteur, les objets et les acteurs avec lesquels il interagit au cours de l'exécution du cas d'utilisation. Dans ce paragraphe, nous présentons quelques diagrammes de séquences permettant de décrire les différentes interactions entre l'utilisateur et l'application.

3.3.1 Diagramme de séquence pour le scénario d'authentification

Avant l'accès à l'application, l'utilisateur doit s'authentifier en introduisant son identifiant et son mot de passe. Après vérification, si l'utilisateur est accepté, il aura accès au système et les informations personnelles de sa session serons prise en compte sinon un message d'erreur s'affichera. Si l'utilisateur tape 3 fois des faux paramètres de connexion, le site le renvoie vers une page de récupération de mot de passe ou bien de quitter le site.

A la réception, le système effectue une vérification de l'authenticité des informations saisies suite à laquelle trois opérations sont envisageables :

- Une session avec le profil de l'utilisateur est crée. Cette session garde trace du parcours de l'utilisateur dans le site.
- Renvoyer l'utilisateur vers la page d'authentification pour retaper de nouveau les informations demandées. L'opération d'identification doit s'effectuer dans un espace sécurisé pour garantir la confidentialité des informations transmises.
- si l'utilisateur tape 3 fois des paramètres de connexion erronés, le site le redirige vers une page sécurisée pour récupérer son identifiant et son mot de passe.

Après l'authentification trois cas se présentent :

- L'utilisateur est un client, le système le redirige vers l'espace des clients.
- L'utilisateur est un employé de la société, le système le redirige vers l'espace des employées.
- L'utilisateur est un administrateur , le système le redirige vers l'espace administratif.

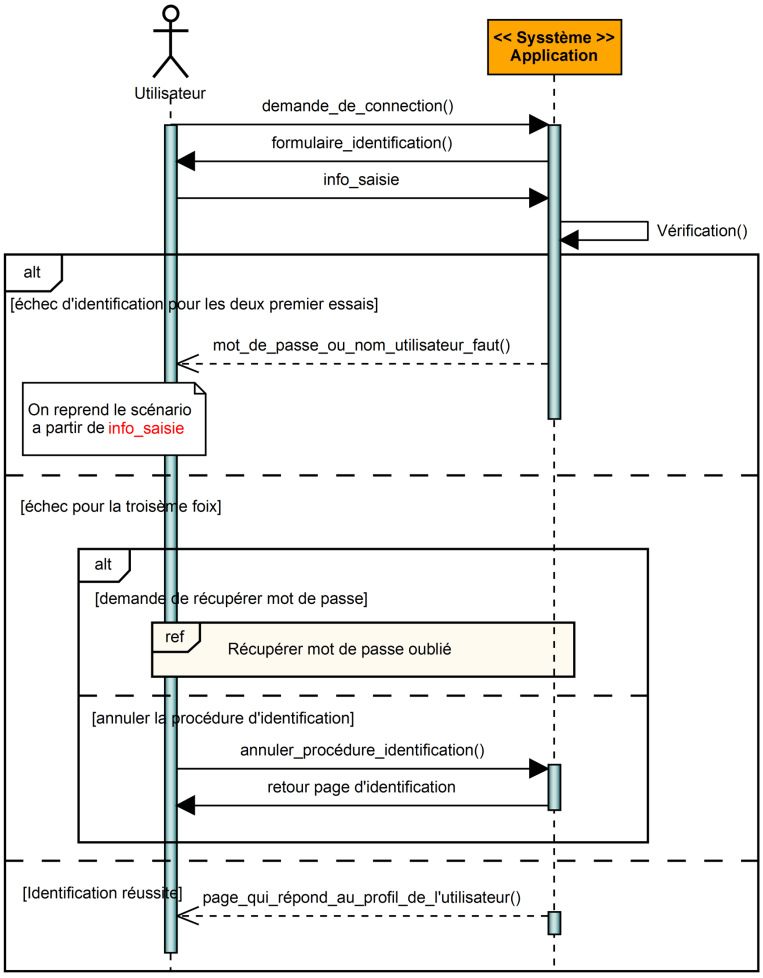


FIGURE 3.12 – Diagramme de séquence pour le scénario d'authentification

L'opérateur « alt » désigne un choix, une alternative. Il représente deux comportements possibles : c'est en quelque sorte l'équivalent du SI...ALORS...SINON : donc, une seule des deux branches sera réalisée dans un scénario donné.

La condition d'exécution d'une des deux branches (l'équivalent du SI) peut être explicite ou implicite. L'utilisation de l'opérateur « else » permet d'indiquer que la branche est exécutée si la condition du « alt » est fausse. [N1]

L'exemple ci-dessus montre un opérateur « alt » :

- Soit il y a un échec d'identification pour les deux premiers essais, dans ce cas là c'est tolérable.
- Soit il y a un échec d'identification pour le troisième essai, dans ce cas là nous passons à la procédure de récupération de mot de passe.
- Soit l'identification réussit et l'utilisateur est redirigé vers son profil.

L'opérateur « ref » référence à un autre diagramme de séquence (c'est un appel de fonction). Nous détaillons par la suite le scénario de récupération du « mot de passe oublié ». Ceci est accessible après trois essais d'authentification non valides. Dans ce cas, l'utilisateur est appelé à saisir son mail, après vérification que cet email existe dans la base, une demande de ressaisir de mot de passe attaché avec un lien sera envoyés directement depuis le serveur vers sa boite mail.

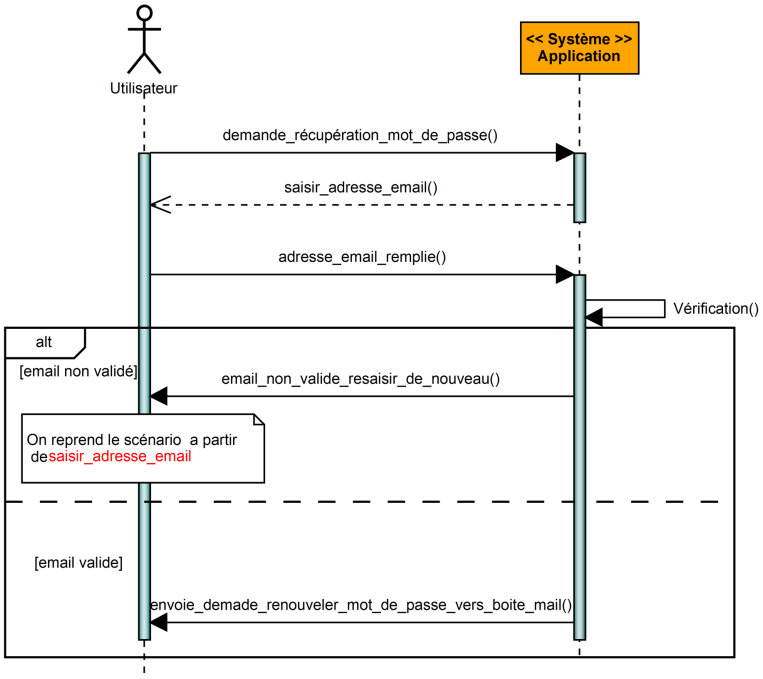


FIGURE 3.13 – Diagramme de séquence pour le scénario renouveler mot de passe

3.3.2 Diagramme de séquence pour le scénario "ajouter employé"

L'administrateur peut ajouter des employés. Pour cela, il remplit un formulaire concernant le nouvel employé et valide. Si les données saisie sont invalide ou l'employé existe l'application renvoie l'administrateur vers un nouveau formulaire d'ajout, sinon l'employé sera inséré correctement dans la base.

L'administrateur peut ajouter de nouveaux utilisateurs (des employés ou des clients). Chaqu'un possède son propre formulaire mais le principe de fonctionnement est le même.

Dans cette partie, nous allons décrire les différentes étapes pour l'ajout d'un employé. Un formulaire à remplir contenant les informations nécessaires à fournir est ainsi affiché.

L'administrateur est appelé à remplir soigneusement ce formulaire et puis valider en appuyant sur le bouton "-OK-"ou bien annuler la tache en appuyant sur le bouton "-Annuler-".

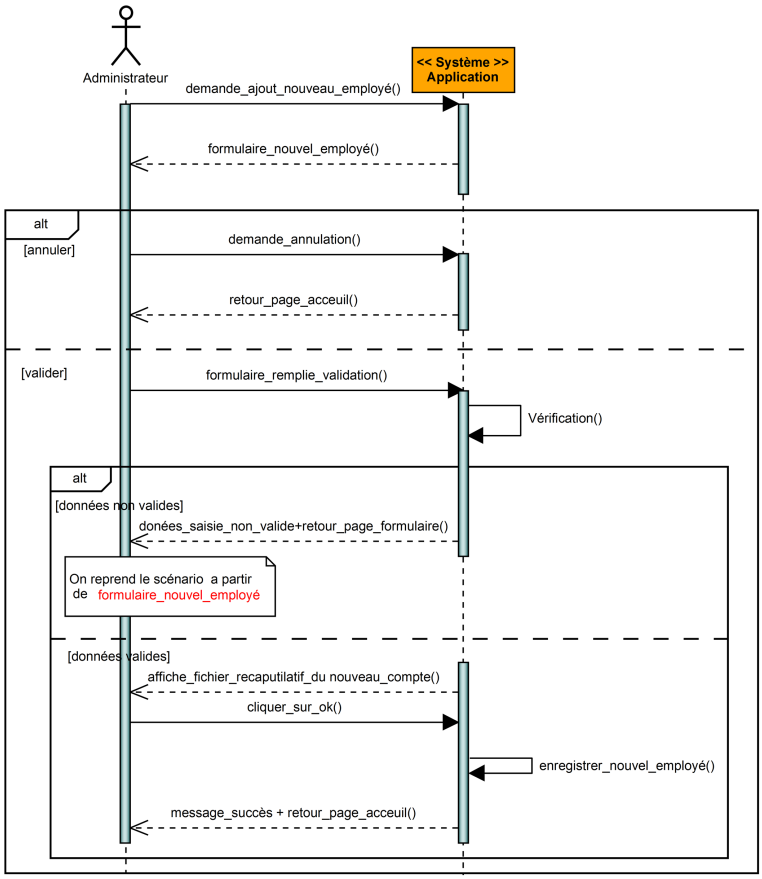


FIGURE 3.14 – Diagramme de séquence pour le scénario ajouter un nouveau compte "employé"

3.3.3 Diagramme de séquence pour le scénario gérer projets

L'administrateur est le super utilisateur de l'application, parmi ces missions la gestion des clients de la société. À travers l'application et après son authentification, il peut accéder à plusieurs fonctionnalités de gestion : Consulter liste des clients, Ajouter compte, Activer/Désactiver compte, etc...

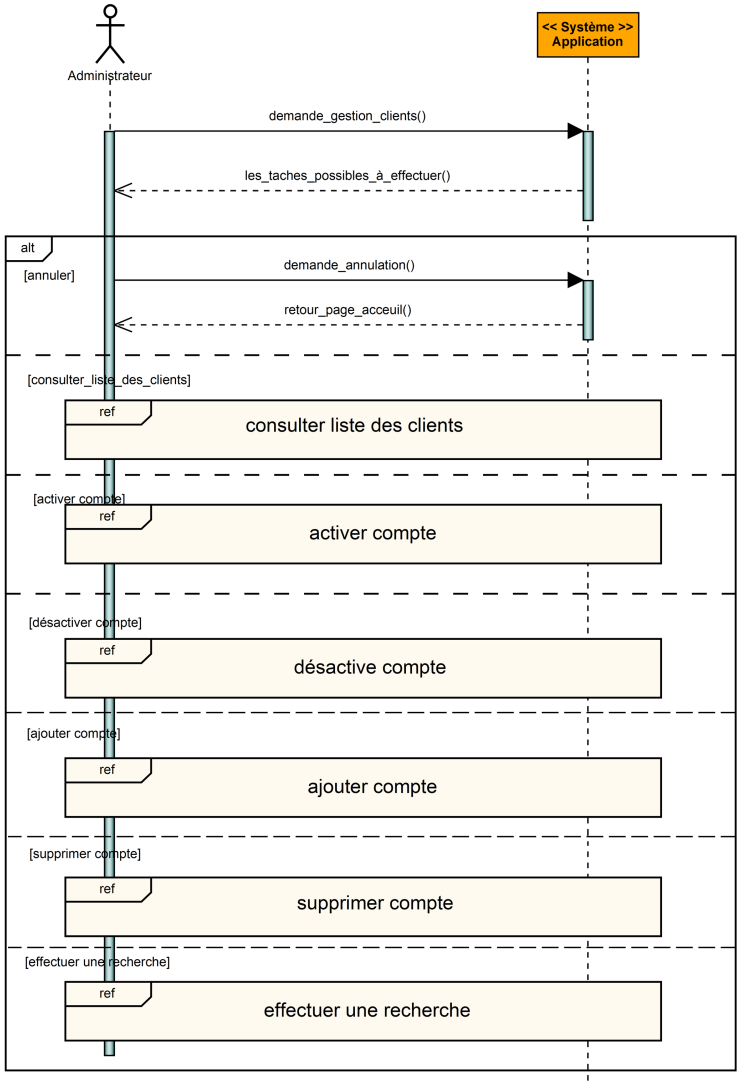


FIGURE 3.15 – Diagramme de séquence pour le scénario gérer clients

3.3.4 Diagramme de séquence pour le scénario d'activation d'un compte

Le client ou l'employé ne peut accéder à l'application qu'après être activé par l'administrateur. Ce dernier consulte la liste des clients ou des employés et vérifie leurs informations

et leurs abonnements dans le site .Et selon le besoin, il choisie d'activer ou désactiver un ou plusieurs comptes. Le schéma ci-dessous présente la procédure d'activation d'un compte pour client.

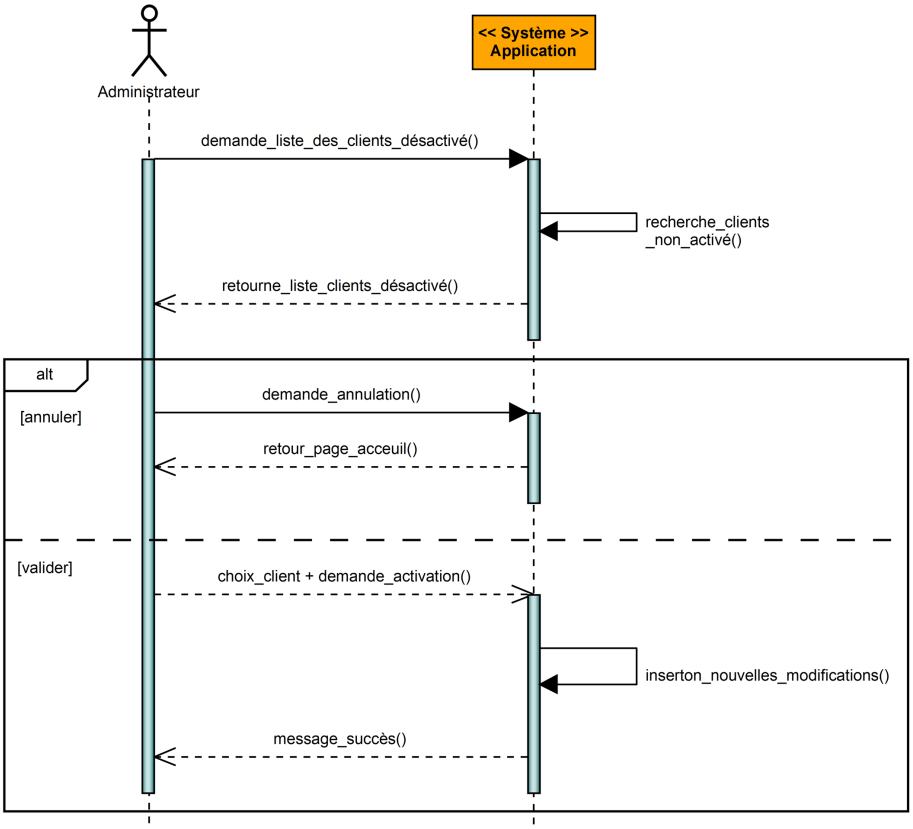


FIGURE 3.16 – Diagramme de séquence pour le scénario activer un compte "client"

3.3.5 Diagramme de séquence pour le scénario de recherche

Le chef de projet aura parfois besoin de chercher un ancien projet. Pour cela, il accède au menu de recherche d'un projet, il choisit un mot clé. Tout de suite, l'ensemble des projets dans la base qui ont ce critère valide seront affichés. Le chef de projet peut alors visualiser les informations détaillées d'un projet, modifier ses information ou ses ressources, gérer ses taches, gérer ses tickets, le supprimer.

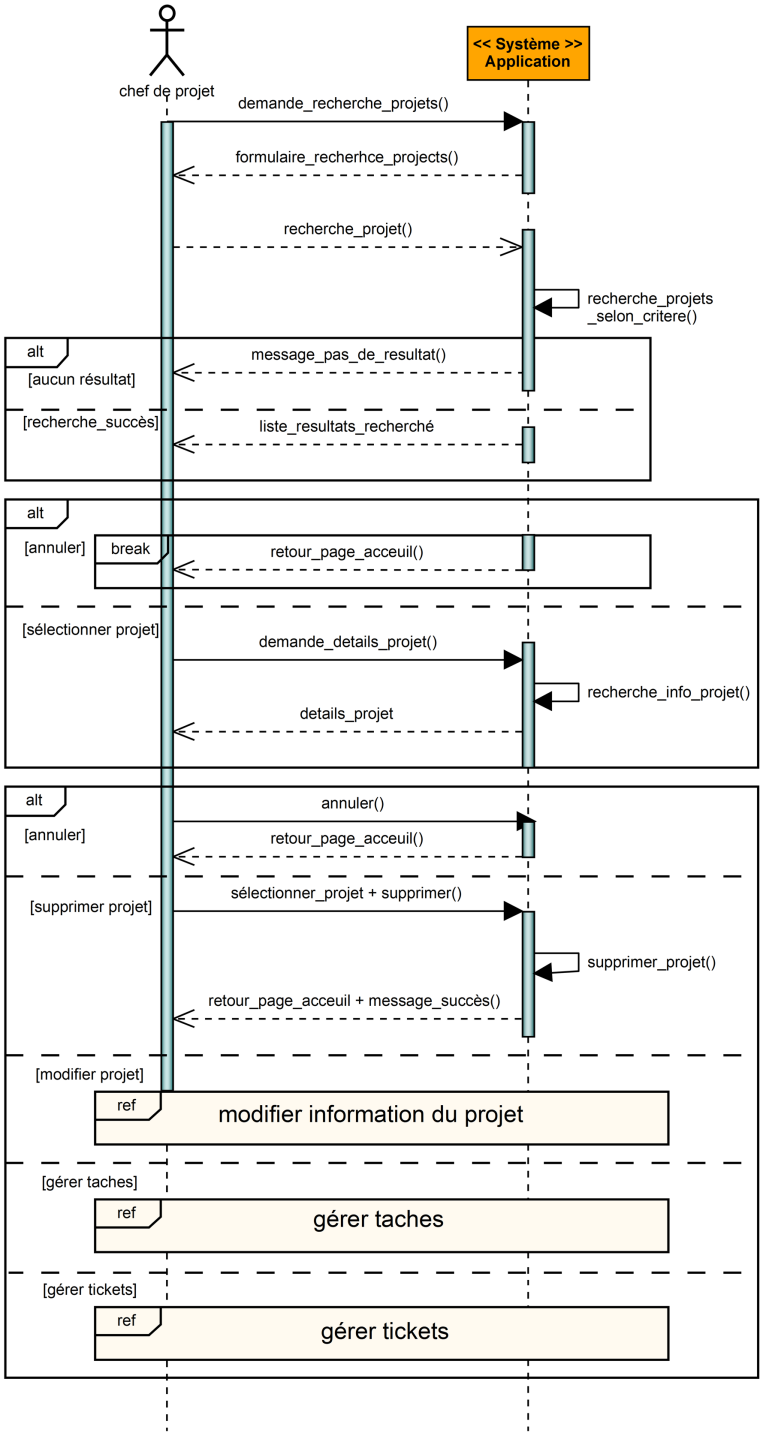


FIGURE 3.17 – Diagramme de séquence pour le scénario recherche projet

L'opérateur "break" est utilisé dans les fragments combinés qui représentent des scénarios d'exception en quelque sorte. Les interactions de ce fragment seront exécutées à la place des interactions décrites en dessous. Il y a donc une notion d'interruption du flot "normal" des

interactions. [N1]

3.3.6 Diagramme de séquence pour le scénario d'ajout d'une nouvelle tache

Le chef de projet, après authentification, peut créer de nouvelles taches sous les projets qui dont il est responsable. En effet, il remplit un formulaire détaillé contenant les date de début et de fin, l'affecter aux employés concernés..., et l'enregistre.

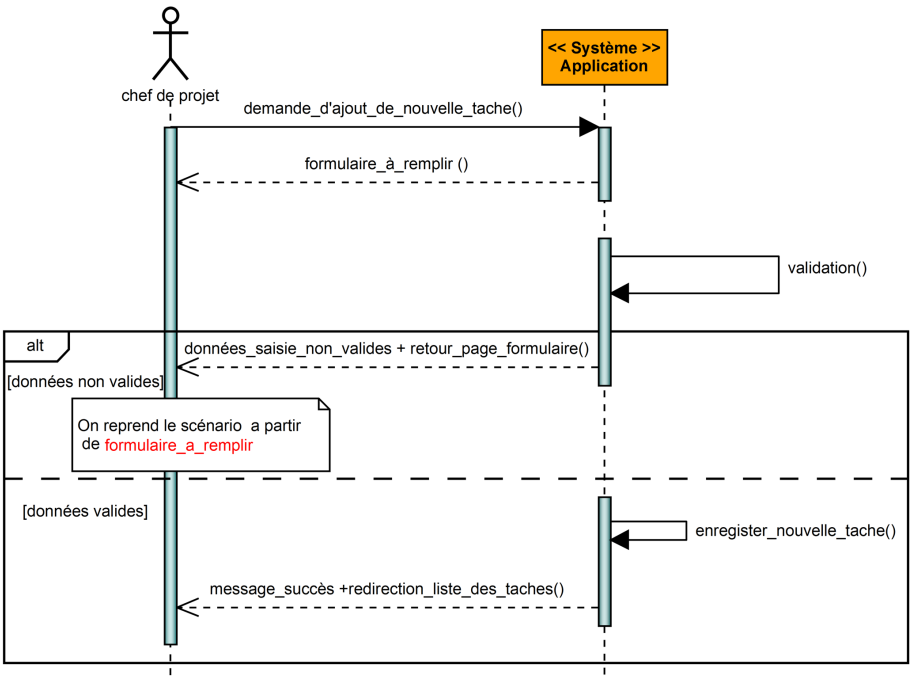


FIGURE 3.18 – Diagramme de séquence pour le scénario ajout nouvelle tache

3.3.7 Diagramme de séquence pour le scénario de validation d'un ticket

Le client, après être authentifier, et si l'administrateur lui a donnée la possibilité, peut créer ou valider un ticket (disfonctionnement, maintenance,..). Le diagramme si dessous, présente la procédure a effectué après le traitement d'un ticket par un employé.

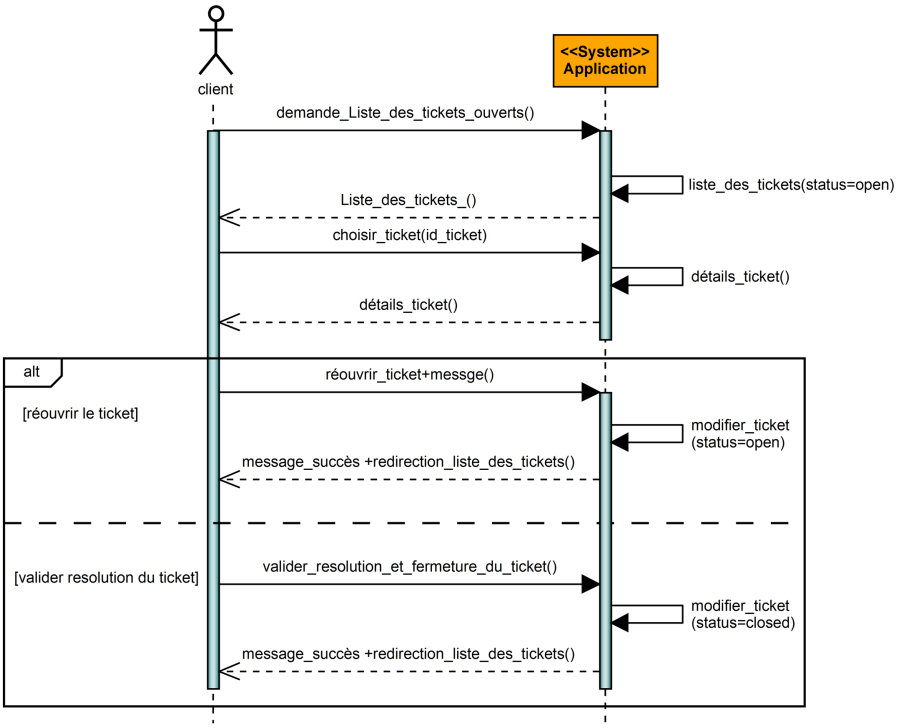


FIGURE 3.19 – Diagramme de séquence pour le scénario de validation d'un ticket

Conclusion

Ce chapitre nous a permis de couvrir tous les cas d'utilisation concernant les différents acteurs de notre application et de définir les besoins non fonctionnels à prendre en considération afin de satisfaire tous les utilisateurs, sans entreprendre l'implémentation. En effet, cette phase donne une vue claire du travail et permet d'initier à la phase qui s'occupe de la modélisation de notre application et qui sera décrite dans le chapitre suivant.

Chapitre 4

Conception

Introduction

Le présent chapitre sera consacré à la présentation de l'étape de conception de notre système. La conception est un processus créatif qui permet de décrire la manière non ambigüe du fonctionnement désiré du système afin d'en faciliter la réalisation et la maintenance. Pour ce faire, nous présentons dans une première partie une conception générale de notre application ensuite dans la deuxième partie nous détaillons la conception de l'application et de la base de données utilisée.

4.1 Conception générale

Notre projet se base essentiellement sur deux grandes parties. La première partie touche au système interne de l'application qui comprend la construction du schéma de la base de données générique. La deuxième décrit l'interface interactive de l'application qui permet à l'utilisateur de spécifier ses données et valider ses choix. Nous justifions, dans le paragraphe suivant, le choix de l'architecture du système.

4.1.1 Conception architecturale

L'architecture de l'application est une architecture trois-tiers basée sur le modèle MVC.

4.1.1.1 L'architecture trois-tiers

L'architecture trois-tiers est un modèle logique d'architecture applicative qui vise à séparer très nettement trois couches logicielles au sein d'une même application ou système, à modéliser et présenter cette application comme un empilement de trois couches, étages, niveaux ou strates dont le rôle est clairement défini.[N2]

- **La présentation des données** :La couche présentation est chargé du traitement de l'interaction avec l'utilisateur. C'est un rôle d'affichage et d'interaction. Elle peut être représentée en HTML pour être exploitée par un navigateur web ou en WML pour être utilisée par un téléphone portable.
- **La couche application** :La couche application effectue de plus le tampon entre la présentation et les données. Elle regroupe les traitements pour simplifier les déploiement et accroître la scalabilité.
- **L'accès aux données persistantes (niveau 3)** : Elle consiste en la partie gérant l'accès aux données du système.L'intégration des règles de gestion fondamentales avec les données pour assurer la cohérence globale et la réutilisation.

Dans cette approche, les couches communiquent entre elles à travers d'un « modèle d'échange », et chacune d'entre elles propose un ensemble de services rendus. Les services d'une couche sont mis à disposition de la couche supérieure. On s'interdit par conséquent qu'une couche invoque les services d'une couche plus basse que la couche immédiatement inférieure ou plus haute que la couche immédiatement supérieure (chaque niveau ne communique qu'avec ses voisins immédiats).

Le rôle de chacune des couches et leur interface de communication étant bien définis, les fonctionnalités de chacune d'entre elles peuvent évoluer sans induire de changement dans les autres couches. Cependant, une nouvelle fonctionnalité de l'application peut avoir des répercussions dans plusieurs d'entre elles. Il est donc essentiel de définir un modèle d'échange assez souple, pour permettre une maintenance aisée de l'application. La figure suivante illustre les trois niveaux de l'architecture.

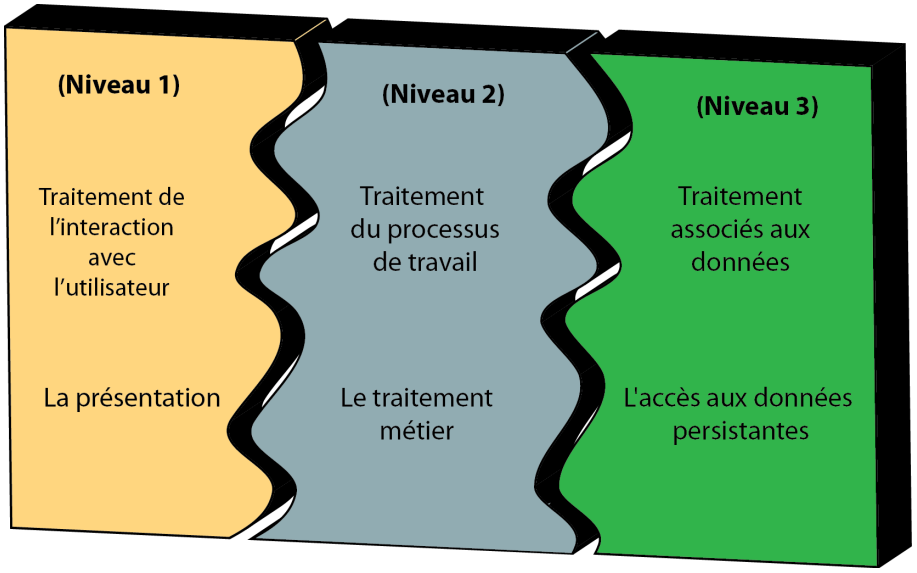


FIGURE 4.1 – Architecture trois-tiers

4.1.1.2 Le modèle MVC

Un Modèle-Vue-Contrôleur (MVC) est une architecture qui organise l'interface Homme-Machine, les traitements et les données d'une application logicielle. [N3]

- **La vue** correspond à l'interface avec laquelle l'utilisateur interagit. Sa première tâche est de présenter les résultats renvoyés par le modèle. Sa seconde tâche est de recevoir toutes les actions de l'utilisateur (clic de souris, sélection d'une entrée, boutons, etc). Ces différents événements sont envoyés au contrôleur. La vue n'effectue aucun traitement, elle se contente d'afficher les résultats des traitements effectués par le modèle et d'interagir avec l'utilisateur.
- **Le Modèle** représente le comportement de l'application : traitements des données, interactions avec la base de données, etc. Il décrit ou contient les données manipulées par l'application. Il assure la gestion de ces données et garantit leur intégrités. Dans le cas typique d'une base de données, c'est le modèle qui la contient. Le modèle offre des méthodes pour mettre à jour ces données (insertion, suppression, changement de valeur). Il offre aussi des méthodes pour récupérer ces données. Les résultats renvoyés par le modèle sont dénués de toute présentation.
- **Le contrôleur** prend en charge la gestion des événements de synchronisation pour mettre à jour la vue ou le modèle et les synchroniser. Il reçoit tous les événements de l'utilisateur et enclenche les actions à effectuer. Si une action nécessite un changement des données, le

contrôleur demande la modification des données au modèle, ce dernier avertit la vue que les données ont changé pour qu'elle se mette à jour. Certains événements de l'utilisateur ne concernent pas les données mais la vue. Dans ce cas, le contrôleur demande à la vue de se modifier. Le contrôleur n'effectue aucun traitement, ne modifie aucune donnée. Il analyse la requête du client et se contente d'appeler le modèle adéquat et de renvoyer la vue correspondant à la demande.

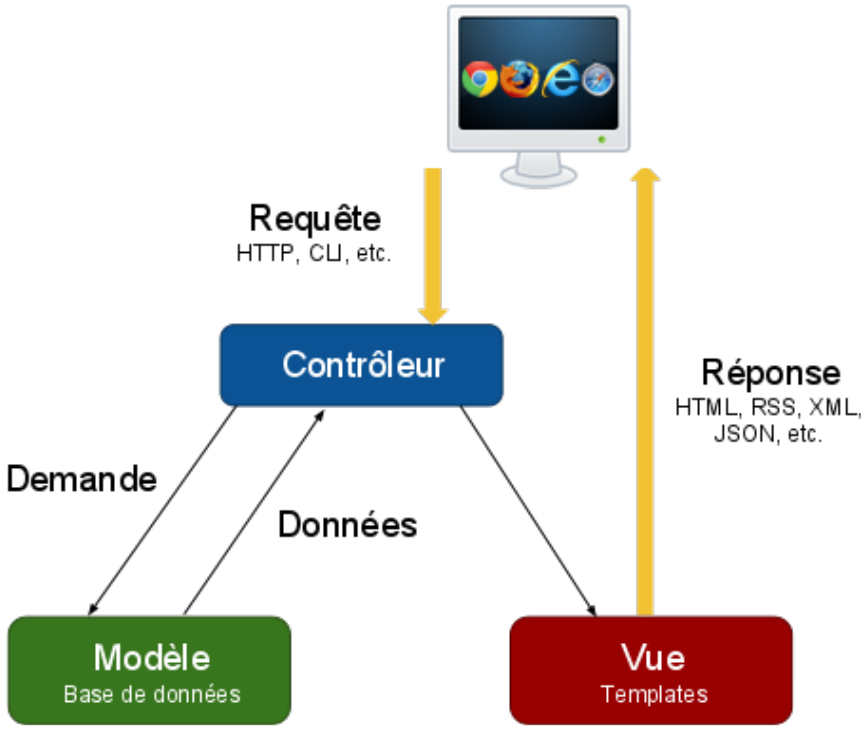


FIGURE 4.2 – L'interaction entre le modèle, la vue et le contrôleur

Comme l'indique la figure ci-dessus, le contrôleur d'après les évènements reçus par l'utilisateur (touches appuyées,...) d'envoyer la vue correspondante. Il effectue la synchronisation entre le modèle et la vue. Il contrôle la modification des données du modèle.

4.1.1.3 Architecture retenue

Notre système offre une interface interactive à l'utilisateur lui permettant de spécifier ses paramètres appropriés à la base de données de la bourse. Afin de faciliter cet aspect générique, nous appliquons l'architecture MVC. Elle sert à bien organiser le système en permettant :

– Une séparation des interfaces, des contrôles des évènements de l'utilisateur et des traitements de données.

– Une synchronisation des différentes vues (interfaces) possibles de l'application.
– Une interprétation plus claire des entrées.
– Une isolation des données (BD, fichiers, etc).

4.2 Conception détaillée

Cette section sera consacrée à la conception détaillée de notre application, nous débutons avec les diagrammes de classes. Puis nous modélisons l'aspect dynamique du système à l'aide des diagrammes de séquence. Ensuite nous présentons une conception de la base de données utilisée. Et finalement nous présentons l'architecture de l'application à réaliser.

4.2.1 Diagramme de paquetage

Un diagramme de paquetages est une collection d'éléments de modélisation statiques (classes, paquetages,. . .) qui montre la structure d'un modèle. Il décrit l'ensemble des classes et leurs associations

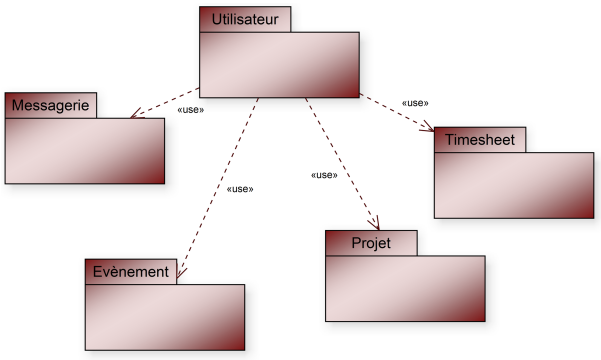


FIGURE 4.3 – Diagramme de paquetage

La figure ci-dessus décrit les différents paquetages de notre application :

– Le paquetage « Utilisateur » contient l'ensemble des classes qui ont une relation directe avec un utilisateur donné. Elle garantie son authentification (paramètres de connexion, statut du compte,..) et ses informations personnelles. Elle contient les groupes des employés et des clients.
– Le paquetage « Projet » contient l'ensembles des classes qui ont une relation directe avec un projet donné.

- Le paquetage « Évènement» contient les classes de traitement des évènements personnels et la planification des réunions.
- Le paquetage « Timesheets » contient les classes de traitement des feuilles de temps, de génération des rapports et des statistiques.
- Le paquetage « Message » contient les classes de traitements des messages et des mails. Il communique avec un serveur smtp avec la bibliothèque swiftmailer qui offre une flexibilité dans l'envoi de courriels riches avec une multitude de fonctionnalité.

4.2.2 Diagramme de classes

Après avoir terminé la phase de conception préliminaire, nous allons maintenant détailler d'avantage cette conception en présentant aussi bien les diagrammes de classes des différents paquetages.

4.2.2.1 Le paquetage « Utilisateur »

Nous allons dans ce qui suit détailler le paquetage « utilisateur » à l'aide d'un diagramme de classes qui permettra de mieux comprendre l'organisation du système. Comme le montre la figure suivante on reconnaît les classes fondamentales pour la bonne gestion des différents utilisateurs du site :

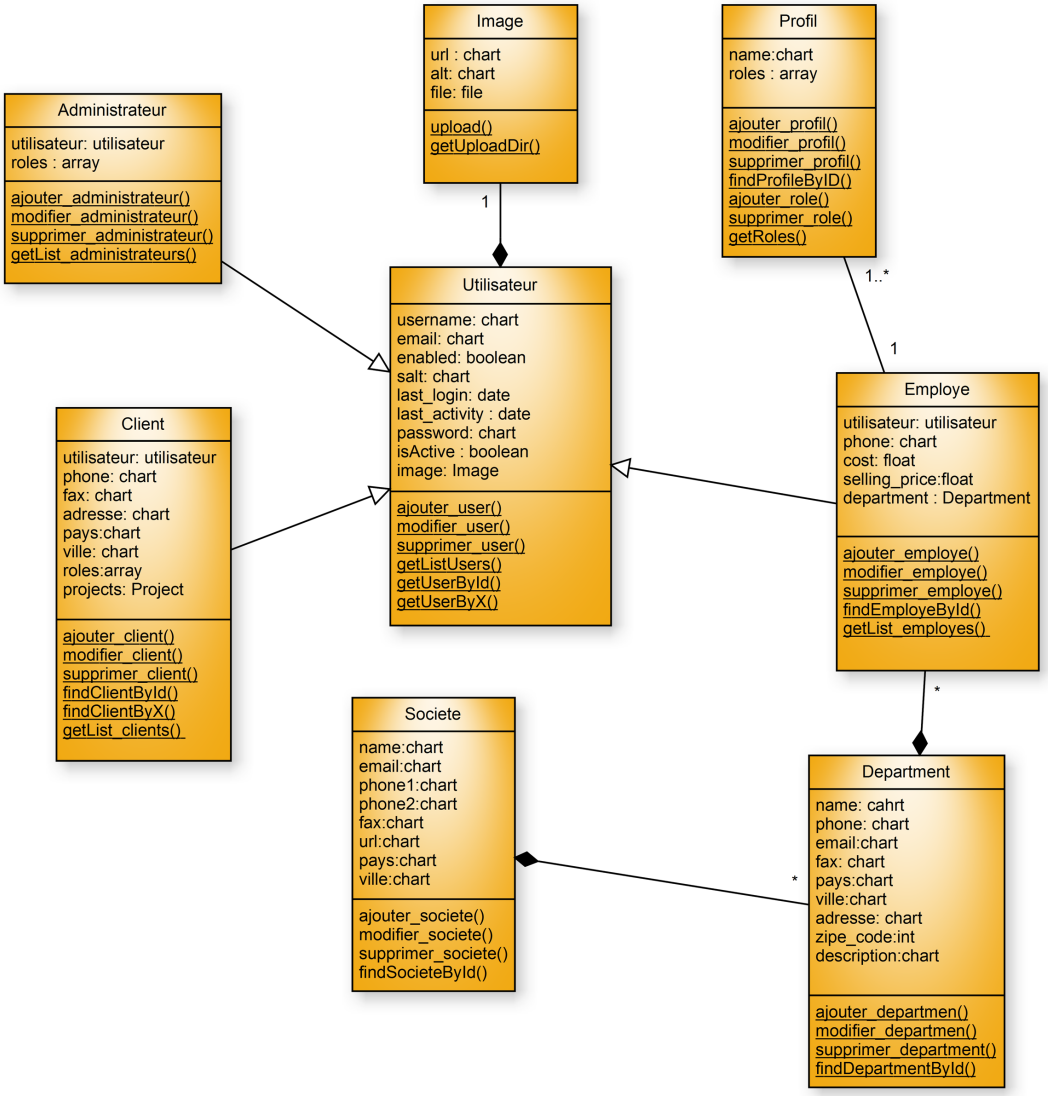


FIGURE 4.4 – Diagramme de classe pour le paquetage "Utilisateur"

La classe Utilisateur : c'est la classe principale qui identifie un utilisateur. Elle contient les informations générales sur lui : username, email, mot de passe, activité, ... etc. Notons que l'attribut salt est utiliser pour hashage de mot de passe.Le hashage consiste en la concaténation d'une ou plusieurs clés (appelées aussi « salt ») au mot de passe, puis le cryptage de la chaine ainsi créée.

La classe Employé : c'est une classe dérivé de la classe utilisateur. Elle nous sert pour définir les employés de la société.

La classe Administrateur : c'est une classe dérivée de la classe utilisateur. Elle nous sert pour définir l'administrateur de l'application. L'attribue rôles désigne l'ensemble des droits d'accès définissant le statut de l'administrateur.

La classe Client : c'est une classe dérivée de la classe utilisateur. Elle décrie décrit une entreprise cliente pour l'application. Un client est le propriétaire d'un projet donné. Notons que l'attribue projet est une instance de la classe projet qui appartient au paquetage Projet (que nous détaillerons par la suite).

La classe Image : est une classe décrivant l'image de profil de utilisateur.

La classe profil : Elle permet à l'employé d'avoir un statut spécifique lors de l'utilisation de l'application. Cette approche assure le regroupement des droits d'accès par lot ce qui facilitera leur gestion (ajout ou suppression d'un ou plusieurs droits)

La classe Département : Elle décrie les départements de l'entreprise.

La classe Société : Elle contient les informations de la société.

4.2.2.2 Le paquetage « Projet »

Dans ce paragraphe nous allons détailler le paquetage le plus important dans l'application : « projet ». Nous reconnaissons à travers la figure suivante les classes nécessaires pour la gestion des projets de l'application (notons que les deux classes Employé et Client ont été définis précédemment dans le paquetage Utilisateur).

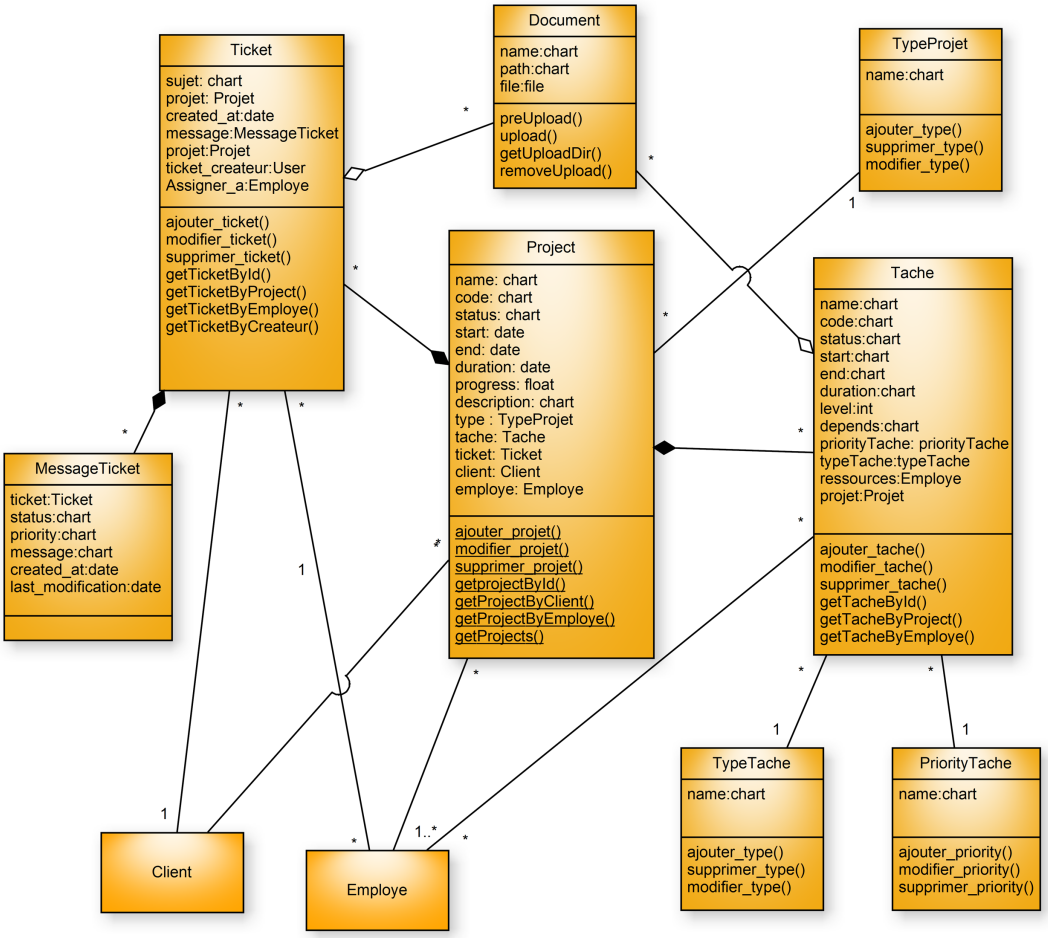


FIGURE 4.5 – Diagramme de classe pour le paquetage "Projet"

La classe Projet : Elle contient les informations décrivant un projet donné (nom, client , date début et fin, progression, status).

La classe TypeProjet : Elle contient la description des type des projets sur lesquels l'entreprise trvaille. Comme notre application se caractérise par son aspect générique, cette approche permet de caractériser le mieux le secteur d'activité de la société.

La classe Ticket : Elle presente le services d'assistance d'un projet donné.

La classe MessageTicket : Elle contient les discution et les echanges d'information pendant la validation d'un ticket.

La classe Document : C'est une classe qui décrie les fichiers jointes utilisés par les taches ou les tickets.

La classe Tache : Cette classe constitue un travail élémentaire du travail à fournir pour produire un résultat. Elle nécessite l'affectation à des ressources humaines (employes) et obei à des contriente de temps ou de dépendaces a d'autre taches

La classe TypeTache : Elle contient la description des type des taches possibles. Comme notre application se caractérise par son aspect générique, cette approche permet une meuilleur définition des taches.

4.2.2.3 Le paquetage « Évènement »

L'application à développer doit fournir un outil fort de planification, non pas au niveau de l'affectation des tache et les calculs des couts seulement, mais au niveau organisationnel pour chaque employé (calendrier personnel, réunions) pour plusieurs objectifs, entre autres nous citons. Dans ce paquetage nous aurons besoin de trois classes, comme le montre la figure suivante. Notons qu'une association plusieurs-à-plusieurs avec attributs est représentée sous UML par une classe-association. Cette classe-association contient les attributs de l'association et est connectée au lien d'association par une ligne en pointillé.[2]

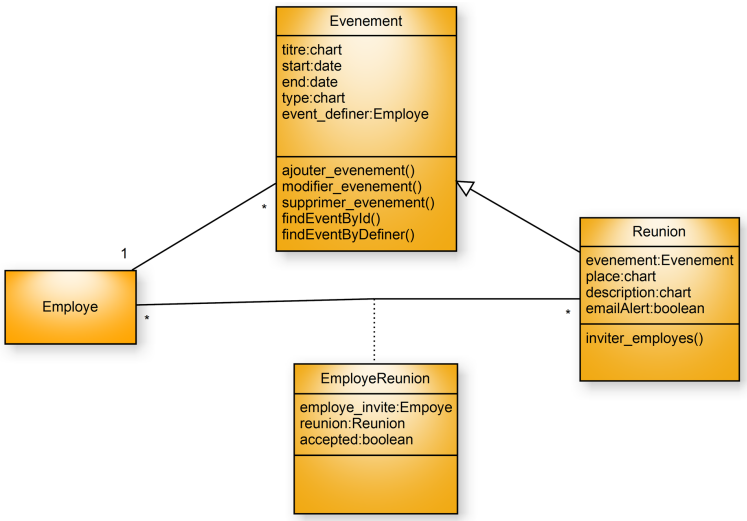


FIGURE 4.6 – Diagramme de classe pour le paquetage "Évènement"

La classe Evenement : l'ensemble des évènements d'un employé donné.Elle présente un moyen d'organisation personnel.

La classe Reunion : est une classe dérivée de la classe Evénement. Elle ajoute

4.2.2.4 Le paquetage « Timesheets »

La gestion des temps est une étape essentielle est un client possédant un ou plusieurs véhicules. Nous avons choisi de la mettre à part pour la bonne gestion des feuille de temps et parce que nous avons besoin pour la génération des rapports et statistiques. Comme le montre la figure suivante on reconnaît la classe fondamentale qu'on peut le nommer tout simplement timesheet (notons que les classes Employe et Tache ont été définis respectivement dans les paquetage Utilisateur et Projet) :

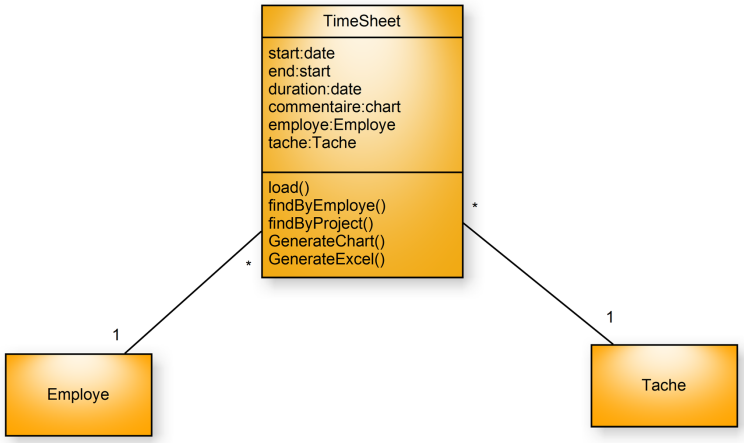


FIGURE 4.7 – Diagramme de classe pour le paquetage "Timesheets"

La classe Timesheet : Elle regroupe les information sur le temps passés dans la réalisation des projets pour les différents employés. Cela est essentiel pour la génération des rapports de suivi.

4.2.2.5 Paquetage « Messagerie »

L'application à développer doit fournir un outil fort de messagerie (envoi et réception instantannée) pour plusieurs objectifs, entre autres nous citons la possibilité pour chef de projet de discuter avec le client ou les développeurs. Dans ce paquetage nous aurons besoin de deux

classes, comme le montre la figure suivante (notons que la classe Utilisateur a été défini précédemment dans le paquetage Utilisateur) :

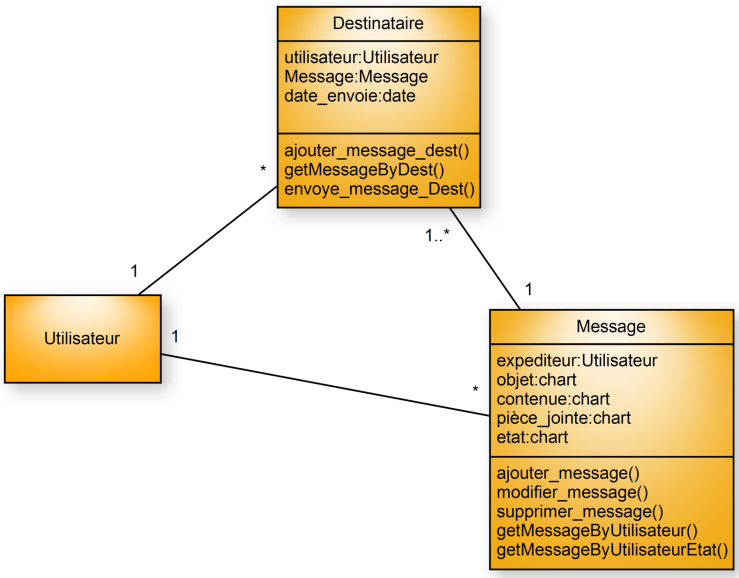


FIGURE 4.8 – Diagramme de classe pour le paquetage "Messagerie"

Un utilisateur peut envoyer plusieurs messages à plusieurs destinataires.

La classe Message : sert pour définir les messages d'un utilisateur donné (objet du message, contenu, pièces jointes, état envoyé reçu ou brouillons).

La classe Destinataire : est une jointure entre la classe Message et la classe Utilisateur. Elle identifie les messages avec leurs destinataires.

4.2.3 Diagrammes de séquences d'objets

Les diagrammes de séquences permettent de représenter des collaborations entre objets selon un point de vue temporel, on met l'accent sur la chronologie des envois de messages.

Dans un diagramme de séquence, un objet est représenté par une boîte contenant le nom de l'objet éventuellement suivi du nom de la classe à laquelle il appartient. Une ligne verticale en pointillés représente la ligne de vie de cet objet.

4.2.3.1 Diagramme de séquences pour le scénario d'authentification

Le diagramme suivant illustre le mécanisme d'authentification des utilisateurs de l'application à savoir les clients, les employés et l'administrateur. Tout utilisateur doit fournir au formulaire de connexion son login et son mot de passe. Une fois que la session est crée (en admettant que le login et le mot de passe sont corrects), le système vérifie les rôles et les fonctions de l'utilisateur connecté, si ce dernier a la fonctionnalité « Administrateur », les liens vers les fonctionnalités d'administration serons ajoutés au menu de la page d'accueil. De même si l'utilisateur est un « employé » ou « client », leurs fonctionnalités respectives seront ajoutées à la session de l'utilisateur connecté.

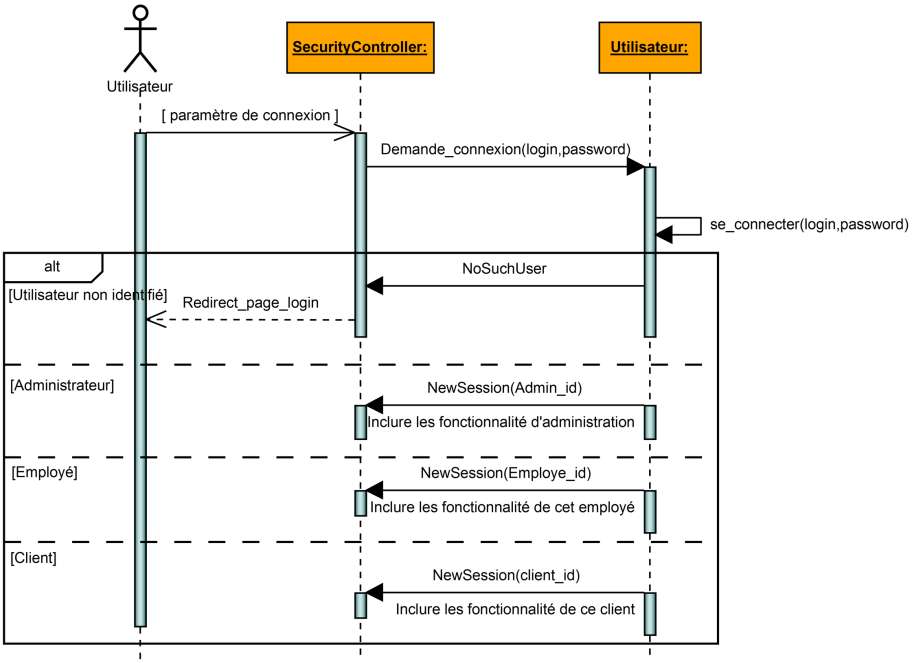


FIGURE 4.9 – Diagramme de séquence d'authentification

4.2.3.2 Diagramme de séquences pour le scénario d'ajout d'un nouveau gestionnaire

Le diagramme de séquences suivant résume le processus de création d'un nouveau profil gestionnaire. L'administrateur remplie le formulaire correspondant et surtout les champs obligatoires. Notre système de validation sera automatique et instantané pour diminuer au maximum les transactions via le serveur. Ainsi un outil de vérification sera actif dans la partie client. Si le formulaire ne contient aucun erreur de saisie, l'administrateur pourra valider sa demande et le nouveau gestionnaire est crée.

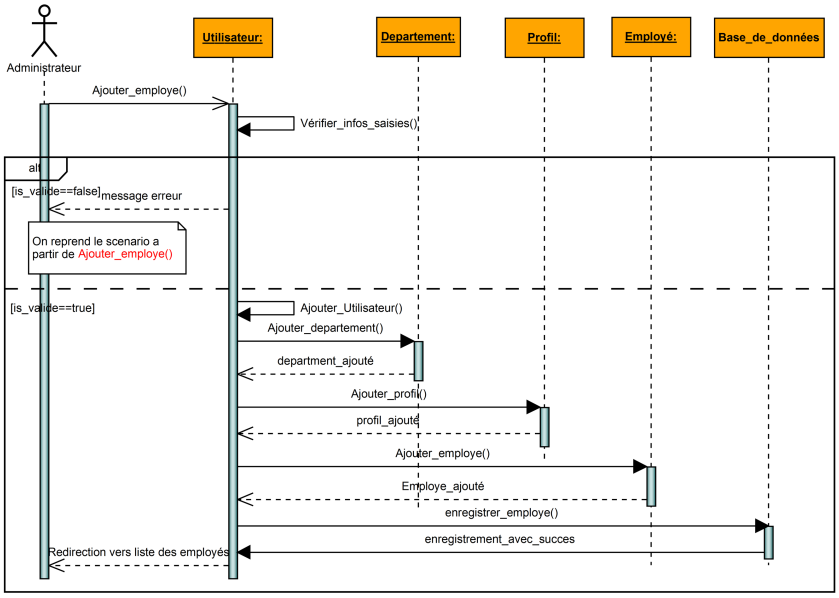


FIGURE 4.10 – Diagramme de séquence pour l'ajout d'un employé

4.2.3.3 Diagramme de séquences pour le scénario d'ajout d'un nouveau projet

Le diagramme de séquences suivant illustre le processus de création d'un projet.L'administrateur remplie le formulaire correspondant et surtout l'affectation des employes, la désignation du client du client propriétaire. Si le formulaire ne contient aucun erreur, la validation s'effectue et le nouveau projet est crée.

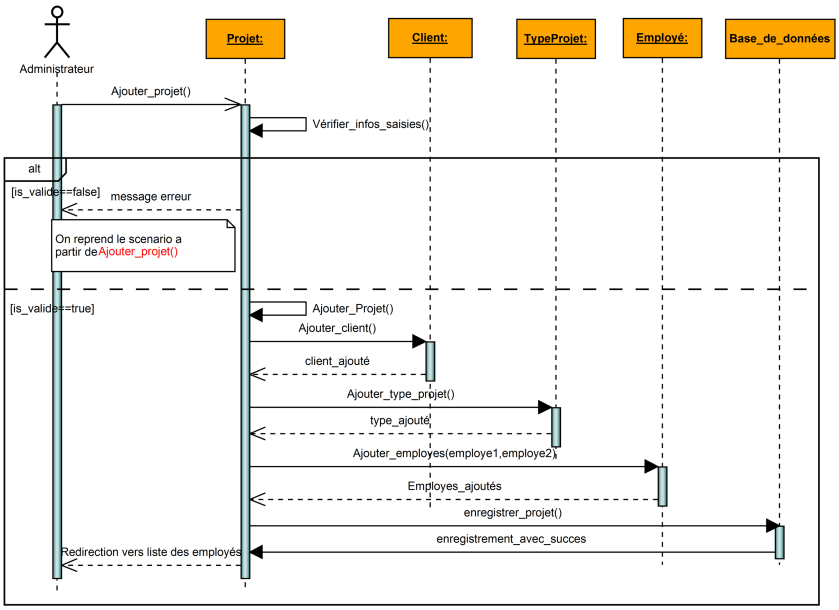


FIGURE 4.11 – Diagramme de séquence pour l'ajout d'un projet

4.2.4 Conception de la base de données

Cette partie est consacrée à la conception de la base de données. En tenant compte des diverses fonctionnalités que l'application doit assurer et afin de garantir l'extensibilité et l'adaptabilité de la base, nous avons conçu un modèle de la base de données relationnelle que nous allons détailler dans ce qui suit.

En tenant compte de la taille de la base, nous avons choisi de présenter un modèle globale montrant les liaisons entre les différentes tables. Ensuite nous présenterons un modèle détaillé montrant les champs dans chaque table.

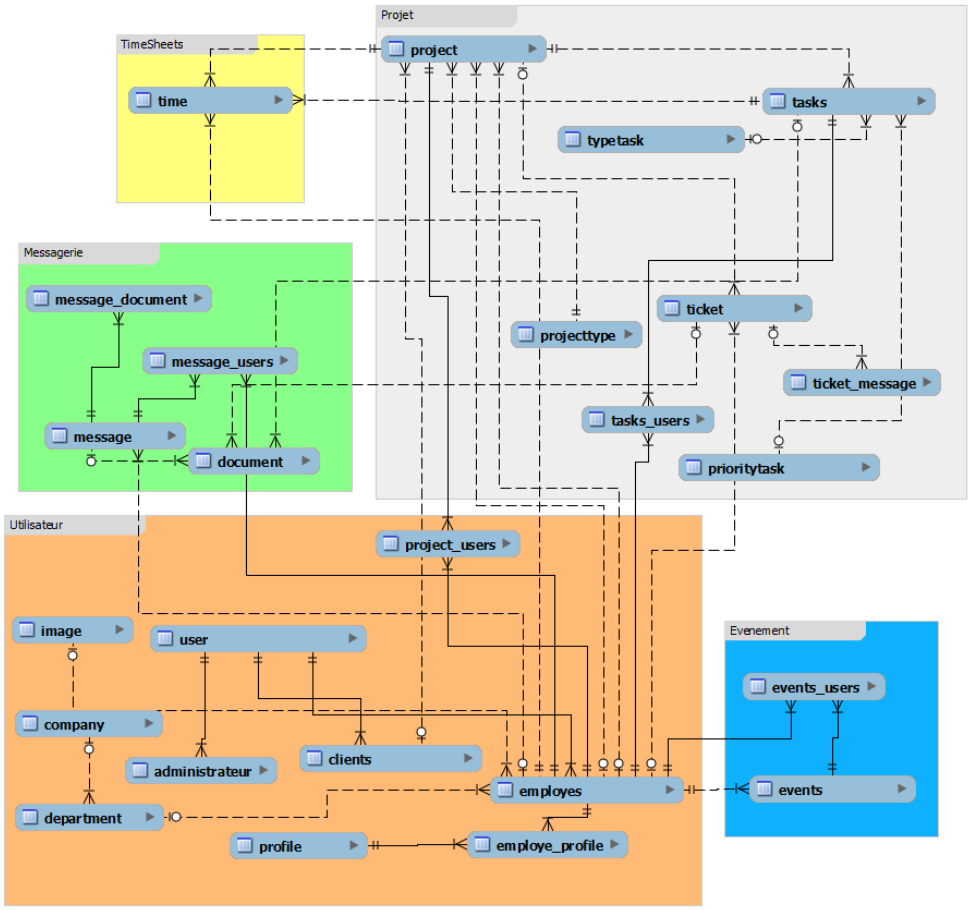


FIGURE 4.12 – Vue globale de la base de données

La figure ci dessus illustre les tables de la base de données de toute l'application. Chaque paquetage est distingué avec une couleur différente. Nous remarquons l'harmonie totale de ce diagramme avec la conception détaillée précédemment. Ce schéma de la base de données lui correspond un modèle plus détaillé montré dans la figure suivante.

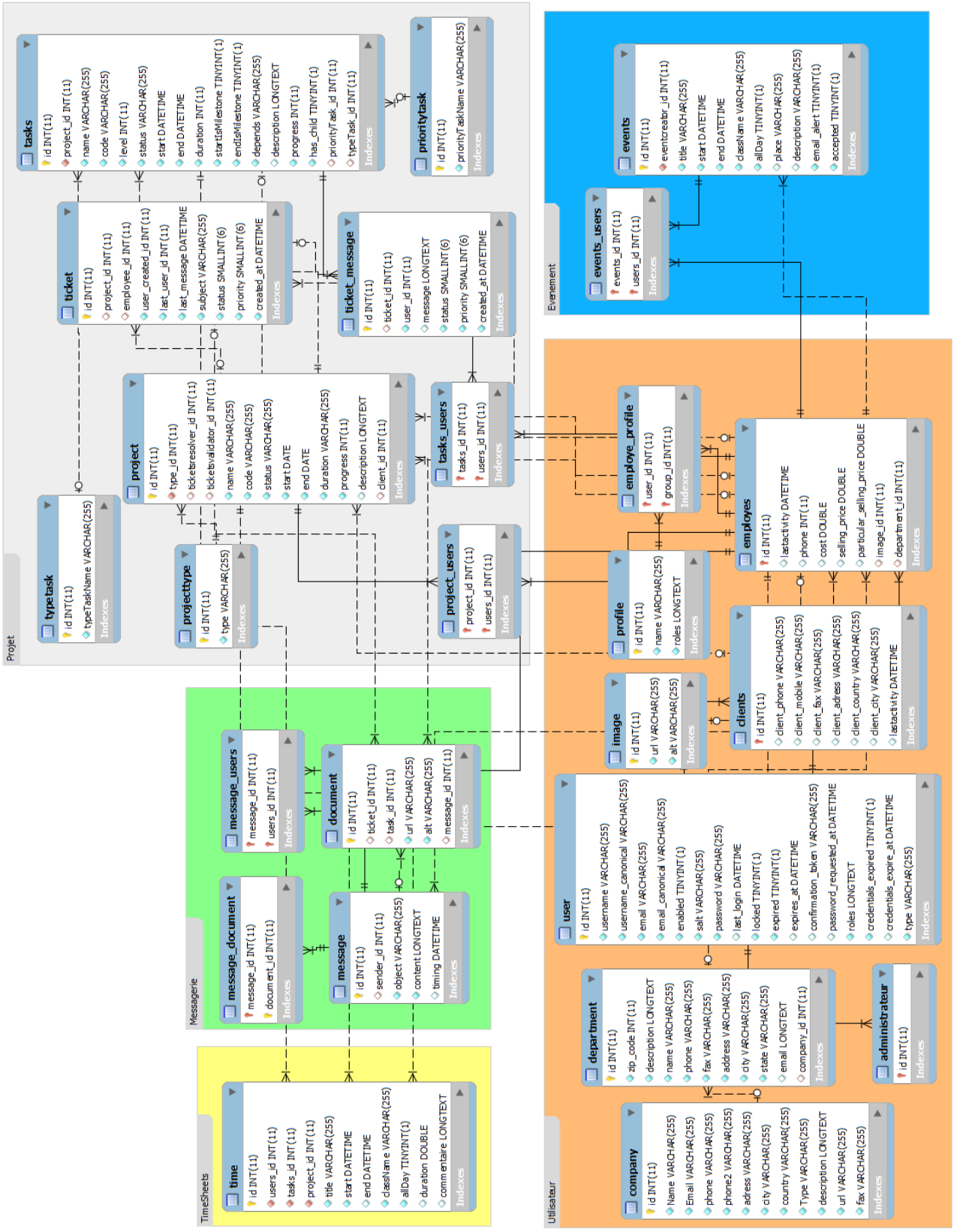


FIGURE 4.13 – Modèle détaillé de la base de données

Conclusion

Tout au long de ce chapitre, nous avons présenté une conception détaillée de notre application. Nous avons commencé par la description de l'architecture globale de l'application, puis nous avons présenté une vue statique de l'application moyennant les diagrammes de classe. Ensuite nous avons élaboré les diagrammes de séquences qui nous ont permis de dégager la vue dynamique de l'application. En fin nous avons pu construire la base de données de l'application avec ses différentes tables.

A présent, nous sommes capables d'entamer la partie réalisation.

Chapitre 5

Réalisation

Introduction

Dans ce paragraphe nous allons décrit sommairement et conformément à la méthodologie SCRUM, les étapes de réalisation de notre projet, ensuite nous allons présenter la réalisation par rapport au produit final en décrivant l'environnement de travail et les technologies utilisées pour la mise en place de notre plateforme. Enfin, nous présentons quelques captures d'écran de l'application réalisée.

5.1 Réalisation par rapport au backlog

La particularité de la méthodologie SCRUM, déjà explicité dans la partie 1.3.3, fait que le déroulement de la réalisation du projet est planifié dés son début. Dans ce paragraphe nous allons décrire brièvement le déroulement des sprints pour ensuite parler du facteur de disponibilité.

5.1.1 Le déroulement des sprints

La décomposition du projet en des sous tâches appelés aussi sprint a été faite au début de notre projet. Il s'agit ensuite, à chaque étape, de faire une réunion d'équipe afin de :

- Enoncer les buts du sprint (le produit du sprint).
- Proposer des estimations temporelles en nombre de jour.
- Valider la réalisation à la fin du sprint en effectuant une démonstration du travail réalisé.
- Modifier éventuellement certains objectifs définis au départ en fonction de nouvelles requêtes.

5.1.2 Facteur de disponibilité

Le facteur de disponibilité, est le rapport du nombre de jour dédié par le développeur pour la participation à un projet et le nombre de jours de disponibilité de ce dernier pendant la période du sprint. Vu que notre participation aux activités de l'équipe est restreinte à notre projet de fin d'étude, notre facteur de disponibilité est élevé de l'ordre de 0.95.

5.2 Environnement de travail

Cette partie présente l'environnement matériel mis à la disposition du présent projet ainsi que l'environnement logiciel que nous avons exploité et nous présentons les choix technologiques en les justifiant.

5.2.1 Environnement matériel

Dans notre projet nous avons développé notre application en utilisant un ordinateur dont la configuration est décrite ci dessous : - Processus Intel Core i3 - Mémoire RAM : 4 Go - Espace Disque : 320 Go

5.2.2 Choix du framework de développement

Le bon choix du framework de développement influe impérativement sur les performances des produits développés. Pour cela nous avons limité notre choix sur les frameworks les plus connus.

5.2.2.1 Les frameworks PHP les plus connus

Le tableau suivant regroupe les frameworks PHP les plus connus.

CakePHP	CakePHP est un framework très simple d'utilisation. Facile à appréhender, il fournit un ensemble de briques de base solides et peut facilement être couplé à un ORM.
CodeIgniter	Code Igniter est un très bon framework qui se fait de plus en plus connaitre de part sa qualité. Il n'embarque pas d'ORM mais dispose de l'essentiel pour répondre à la plupart des demandes.
Zend	Le ZendFramework est plus considéré comme une grosse bibliothèque de fonctionnalités plutôt qu'un framework à part entière et n'est pas très simple à comprendre pour un débutant. Dans sa version 2, il passe réellement dans la catégorie des frameworks avec une approche très proche de celle de Symfony2. Néanmoins, de solides connaissances sont aussi à prévoir afin d'exploiter correctement l'outil.
Symfony2	Symfony2 est un framework qui dispose d'un bon nombre de briques de base tout en laissant pas mal de libertés au développeur. Cependant, Symfony2 peut être délicat à aborder pour un débutant s'il n'a pas de bonnes connaissances en programmation orientée objet et une certaine maîtrise des technologies web est nécessaire pour en exploiter toute la puissance du produit.

TABLE 5.1 – Les frameworks PHP les plus connus

5.2.2.2 Le framwork de développement Symfony 2

Nous avons eu recours dans le cadre de ce projet à la plateforme Symfony qui vise à simplifier le développement des applications web.

Symfony est un framework PHP développé par une équipe française dont le chef d'orchestre est Fabien Potencier. Son but est l'accélération du développement et de la maintenance d'applications web, en se basant sur le modèle maintenant classique : Model-View-Controller. Il est gratuit et open source [N4].

L'idée de base de Symfony semblerait être "ne pas réinventer la roue". En effet le framework consiste à assembler divers projets reconnus existants avec leurs spécialités propres, en les liant dans une même structure de développement "tout en un".

5.2.3 Environnement logiciel

Nous consacrerons cette partie à la représentation des différentes technologies et outils logiciels utilisés pour le développement de notre application.

5.2.3.1 La modélisation UML

Pour répondre à nos besoins et à nos exigences dont les plus importants : la réutilisabilité, la modularité et l'extensibilité, nous avons choisi de modéliser avec le langage de modélisation UML en utilisant l'outil de design « Pacestar UML Diagrammer ».

5.2.3.2 IDE Eclipse Kepler 4.3

Dans ce paragraphe nous allons faire la présentation de l'environnement de développement que nous avons utilisé au travail : Eclipse est un IDE (environnement de développement intégré) écrit en Java, extensible par des greffons (plugins), multi-langages et multi-plates-formes. Il est d'abord conçu pour le langage Java mais ses nombreux greffons en font un environnement de développement pour de nombreux autres langages de programmation (C/C++, Python, PHP, Ruby, ..). [N4]

Toutes les fonctions qu'on peut attendre de ce genre de logiciel sont présentes ou existent sous forme de greffons .En effet, Un point intéressant est son intégration d'outils de gestion de versions comme Git que nous utilisons tout le temps. Ce qui permet de travailler facilement à plusieurs sur un même projet. Eclipse est aussi très bon dans l'écriture de code PHP, avec une bonne coloration syntaxique, la documentation des méthodes sous forme de pop-up, l'auto-complétion des variables, classes, méthodes. Il peut générer automatiquement vos getters et setters. L'ensemble de votre projet est à chaque fois parsé, ce qui permet à l'éditeur de mieux voir les erreurs de syntaxes notamment. Enfin, Eclipse est très bon dans les tests unitaires, il gère aussi l'envoi du projet sur un serveur distant (FTP, SFTP), il peut débugger du PHP en utilisant xdebug.

5.2.3.3 Autres outils

Parmi les autres outils que nous avons utilisés, nous citons :

– Filezilla : un client FTP libre et simple d'utilisation qui permettra de se connecter à distance sur un serveur afin d'y télécharger des fichiers.

- Firebug : Firebug est un outil de développement web sous forme d'une extension pour Mozilla Firefox et SeaMonkey qui permet de déboguer, modifier et contrôler le HTML, le CSS, le DOM, le XHR et le JavaScript d'une page web.
- Composer : Composer est un gestionnaire de dépendance du niveau applicatif pour le langage de programmation PHP qui fournit un format standard pour la gestion des dépendances de logiciels PHP et des bibliothèques requises.
- Msysgit : msysGit est l'environnement de développement pour compiler Git pour Windows. Git est un logiciel de gestion de versions décentralisé.
- Apach server : est un serveur HTTP créé et maintenu au sein de la fondation Apache.
- MySQL WorkBench : est un logiciel de gestion et d'administration de bases de données MySQL. Via une interface graphique intuitive, il permet, entre autres, de créer, modifier ou supprimer des tables, des comptes utilisateurs, et d'effectuer toutes les opérations inhérentes à la gestion d'une base de données.
- Latex : pour la rédaction du rapport.

5.3 Travail réalisé

Les interfaces Homme/Machine constituent un élément important dans la réussite d'une application. Cette contribution est d'autant plus importante lorsqu'il s'agit d'une application Web. Ainsi, nous essayerons dans cette partie de présenter les interfaces les plus signicatives de notre application.

5.3.1 Interface d'authentification

Au lancement de l'application, l'utilisateur se trouve devant une fenêtre d'authentification présenté dans la figure suivante. Cette interface est commune pour tous les utilisateurs : administrateur, employés et clients.

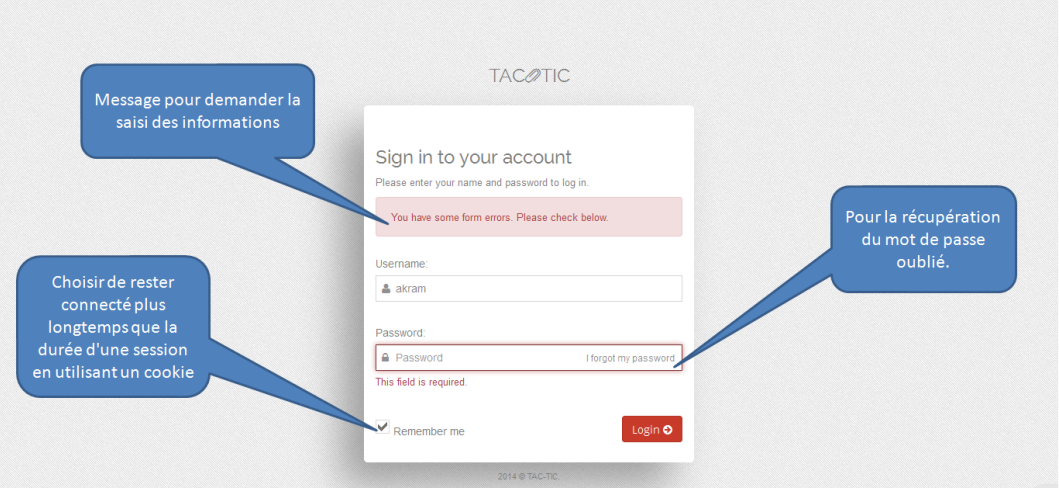


FIGURE 5.1 – Interface d'authentification

5.3.2 Interface d'accueil

Une fois authentifié, l'utilisateur se trouve dans son espace d'accueil. Cet espace est personnalisable selon le type d'utilisateur (dans le cas d'un administrateur / employé ou client des liens vers les fonctionnalités spécifique pour chaque utilisateur s'ajoute au menu principal). La figure suivante présente l'interface d'accueil avec toutes les fonctionnalités offerte à un employé ayant un profile de chef de projet.

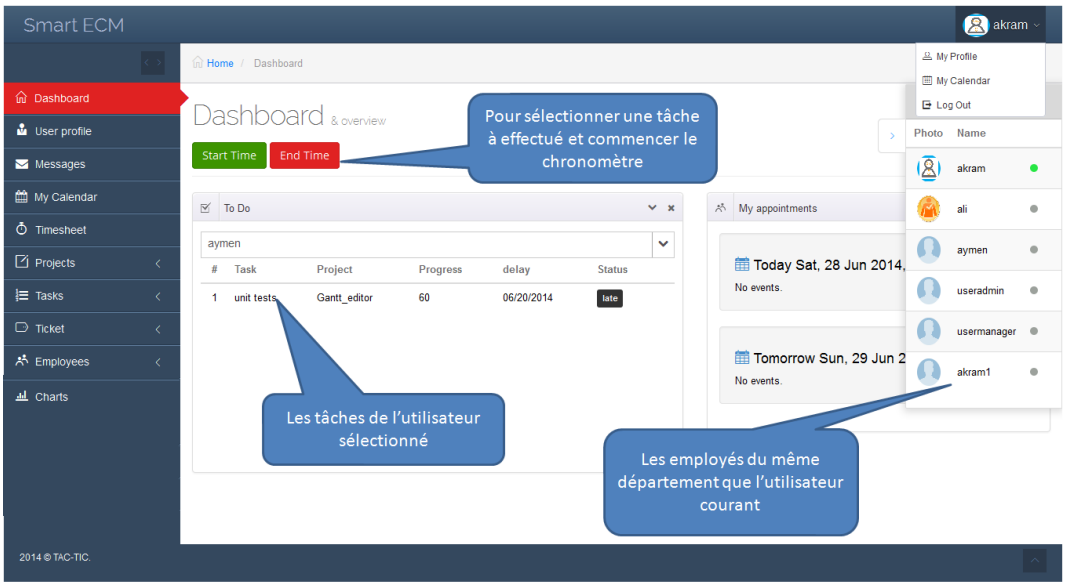


FIGURE 5.2 – Page d'accueil pour un chef de projet

5.3.3 Interfaces tous les projets

Cette fenêtre regroupe tous les projets auxquels participe l'utilisateur courant. Elle donne un aperçu global sur chaqu'un (client, date de début, date de fin, statut courant). Notons que les onglés de modification ou de suppression ne sont visible que pour l'administrateur. La figure suivante illustre la liste de tous des projet de la société.

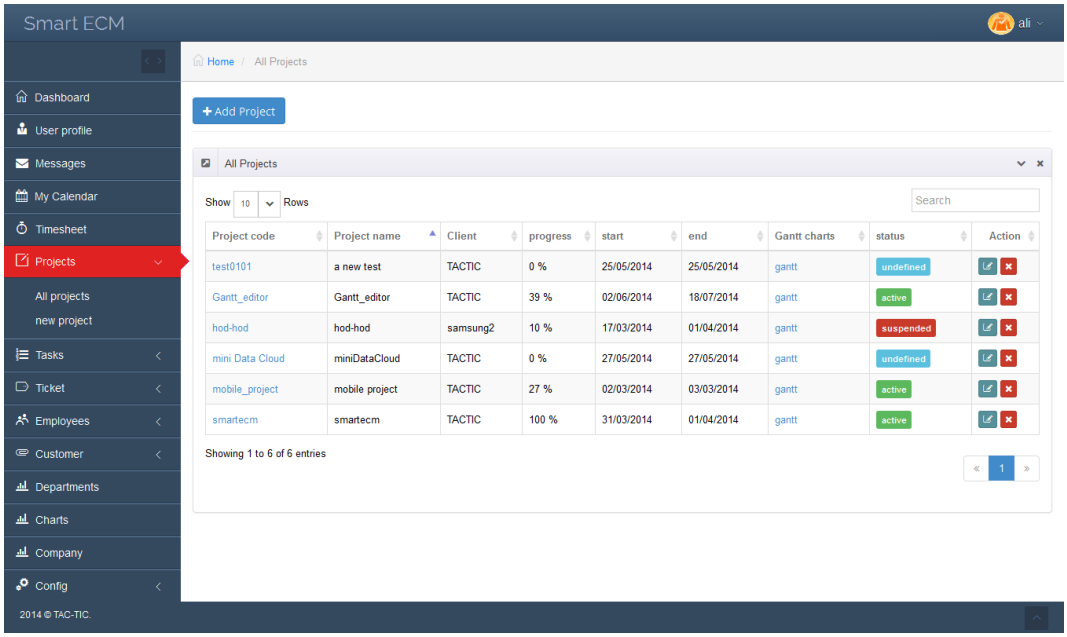


FIGURE 5.3 – Tous les projets

5.3.4 Interface Ajout d'un projet

Pour ajouter un nouveau projet, l'administrateur doit remplir le formulaire représenté par la figure 5.4. Pour cela, il doit choisir le client, le type de projet, les employés comme il doit désigner le validateurs par défaut des tickets sous ce projet.

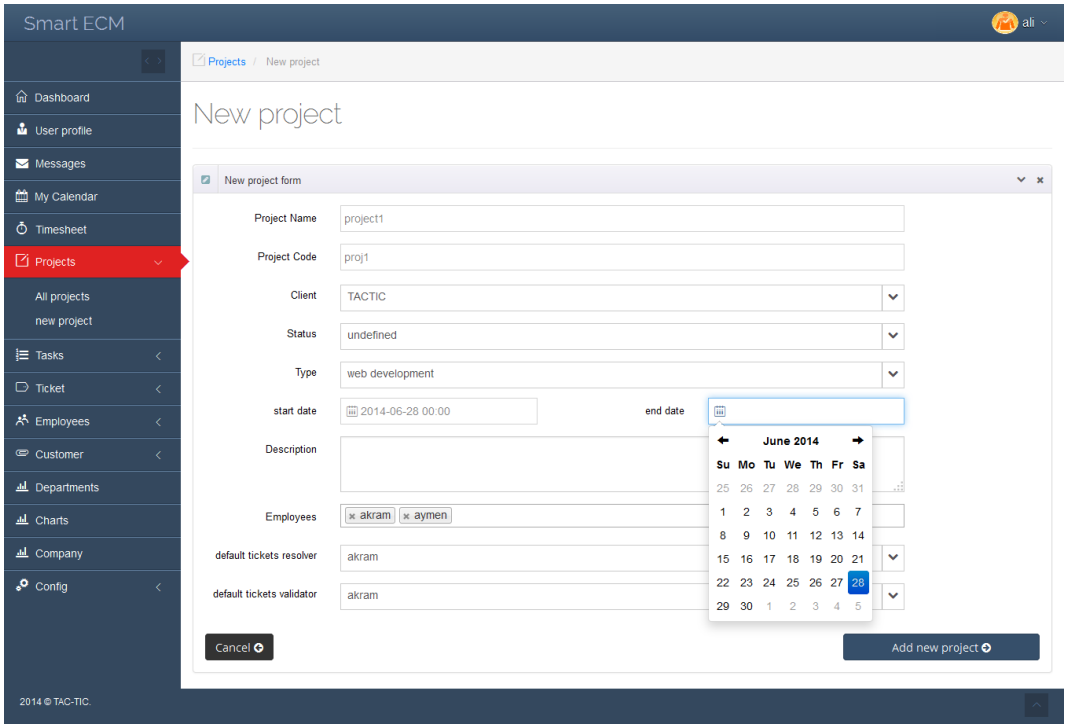


FIGURE 5.4 – Ajout d'un projet

5.3.5 Interfaces suivre un projet

En choisissant un projet, une page d'écrivant toutes les informations (employées, tâches, tickets) est affichée (figure 5.5).

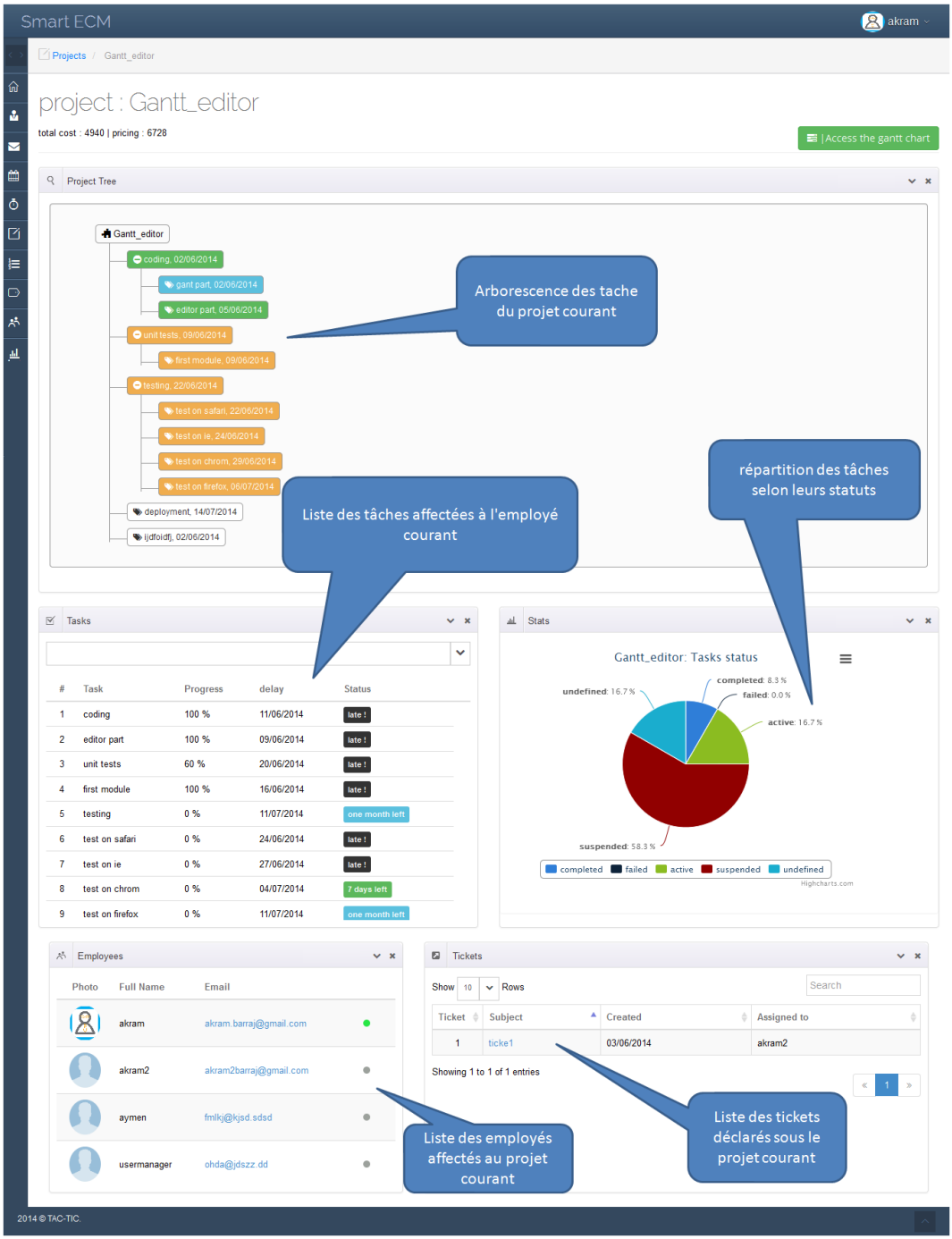


FIGURE 5.5 – Page détails d'un projet

5.3.6 Interfaces diagramme de gantt

Le diagramme de gantt utilisé dans cette application est un peut évolué.En effet, dans la phase de planification le chef de projet, en utilisant les outils offertes, créer et asségnier les taches, ajouter les dépendances et avoir une apperçu sur les couts en terme d'investissement en ressources humaine. Dans la phase de réalisation, il peut suivre la progression des taches affeté au différents employés. La figure 5.5 illustre un projet en phase de réalisation.

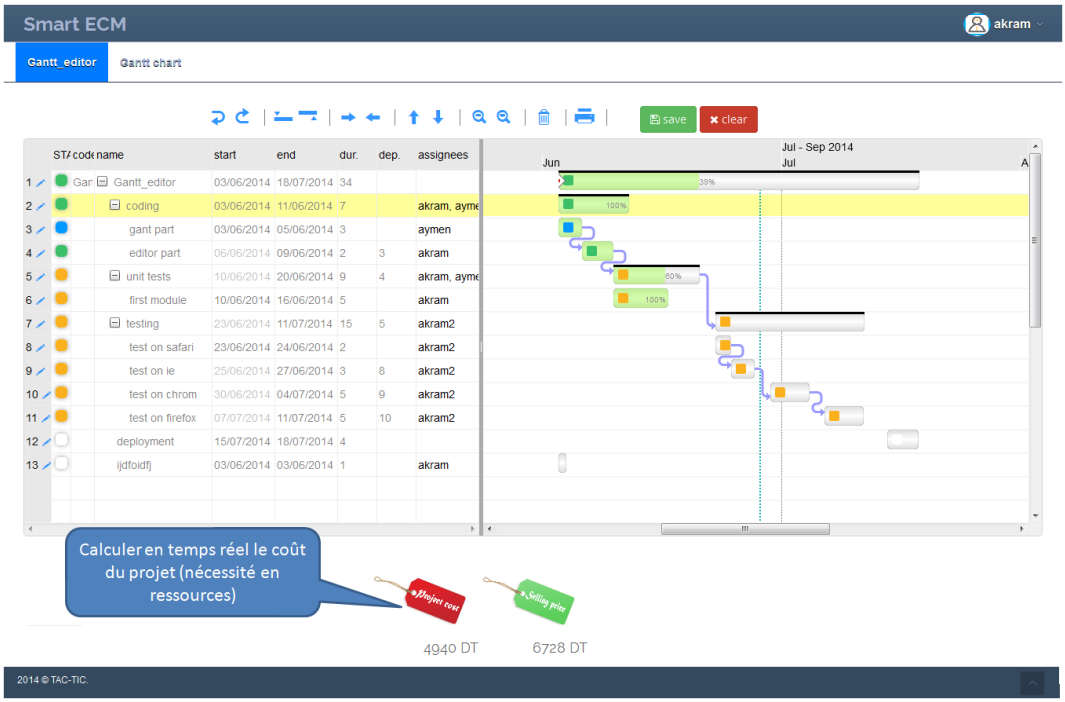


FIGURE 5.6 – Page du diagramme de gantt

5.3.7 Interfaces de messagerie

La figure 5.6 montre la boite de messagerie de l'utilisateur courant. Les employés peuvent envoyer un message a un ou plusieurs destinataire avec la possibilité d'attacher des documents.

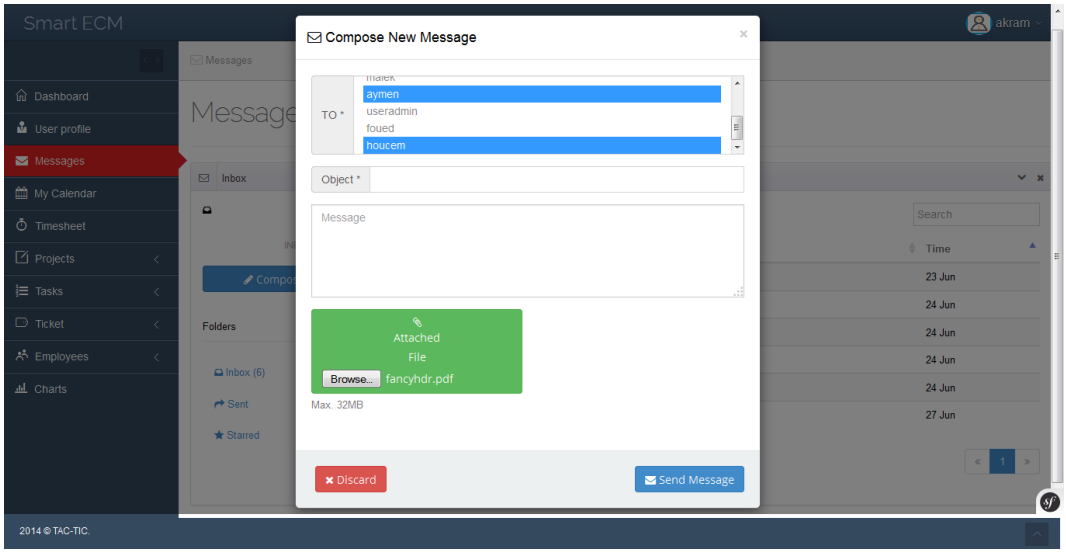


FIGURE 5.7 – Envoyer un message

5.3.8 Interfaces du calendrier

Chaque employé de la société a un calendrier personnel (figure 5.7) avec lequel il planifie ses évènements et reçoit les notification sur les reunions avec les autre membres.

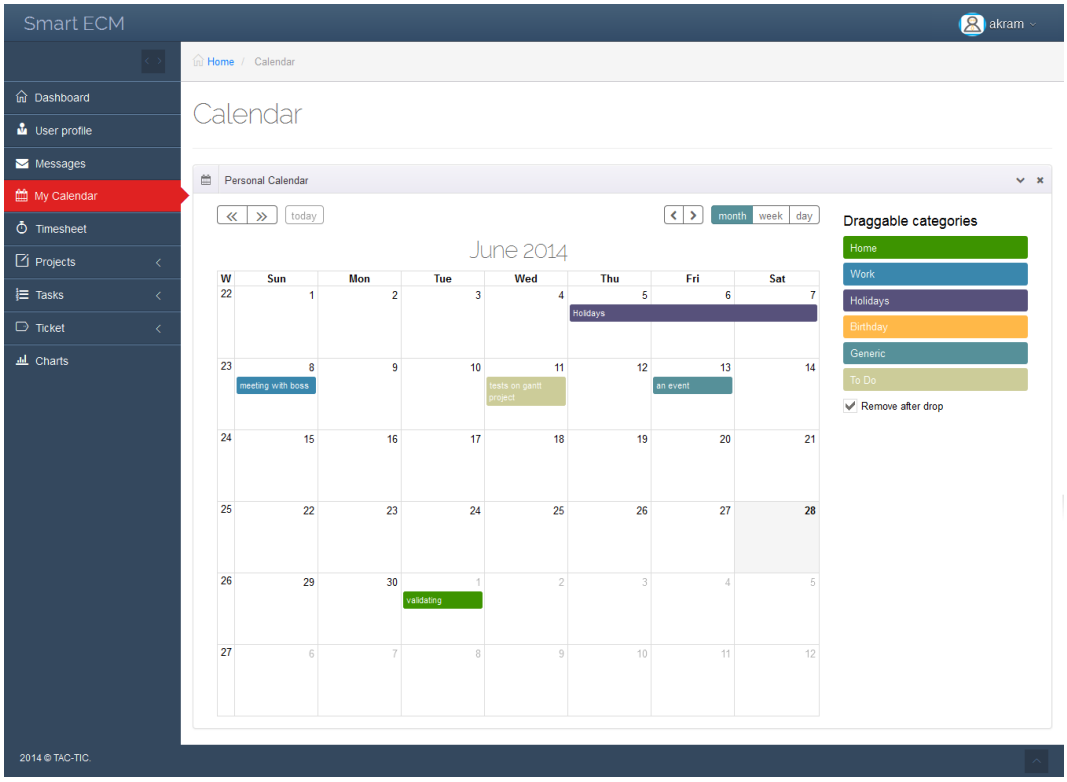


FIGURE 5.8 – Le calendrier personnel

5.3.9 Interfaces suivi d'un ticket

Pour signaler une nouvelle demande , le client doit réserver un ticket en saisissant une description détaillée.

Aprés la déclaration, le client aura la possibilité de suivre l'évolution de la nouvelle tache en ajoutant des commentaire (figure 5.9). Cela permettra une meilleure experience clientèle avec des résultats plus satisfaisantes.

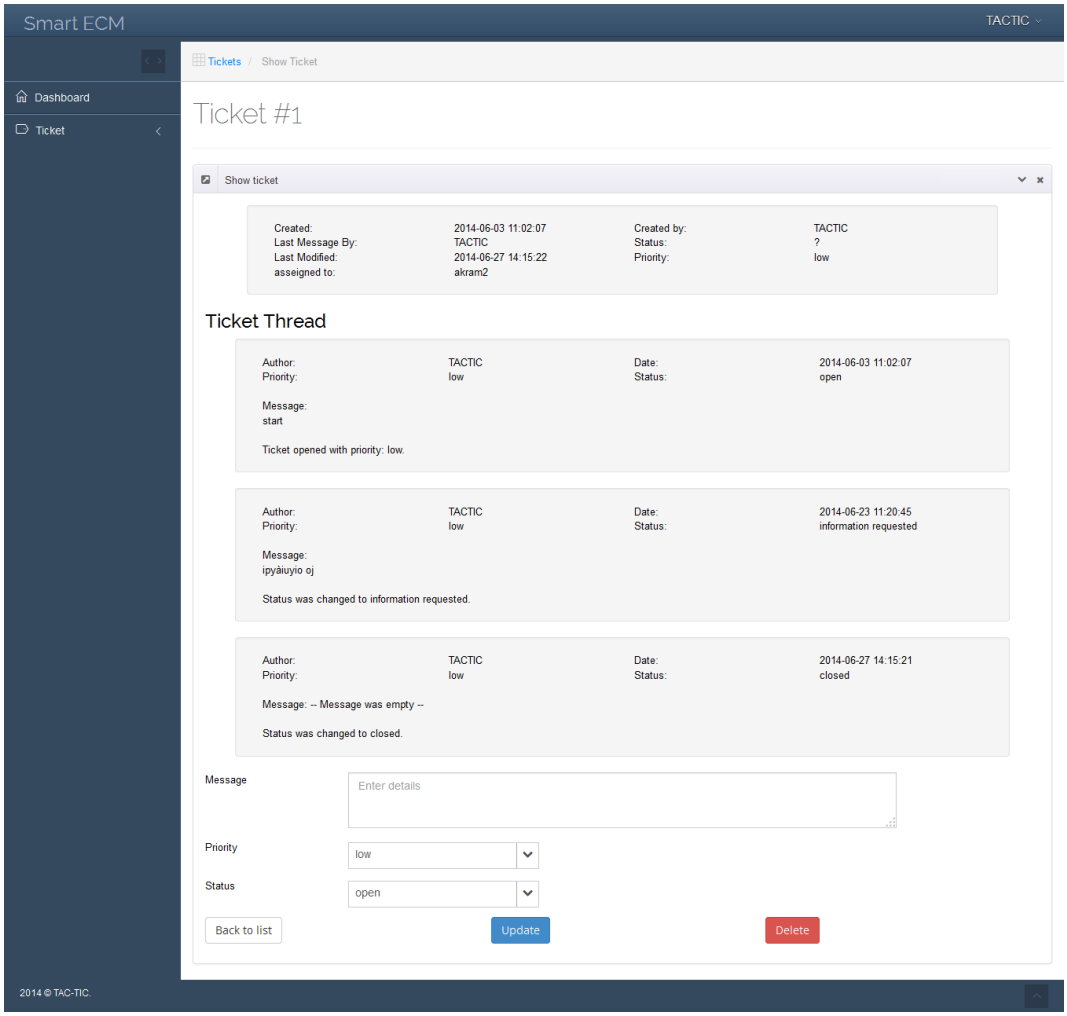


FIGURE 5.9 – Suivi d'un ticket

5.3.10 Interfaces statistiques

Pour consulter les statistique de réalisation des projets, le chef de projet remplit un formulaire pour il choisira : les employés et les projets. La figure 5.10 illustre le temps accomplie pour

chaque employé par projet.

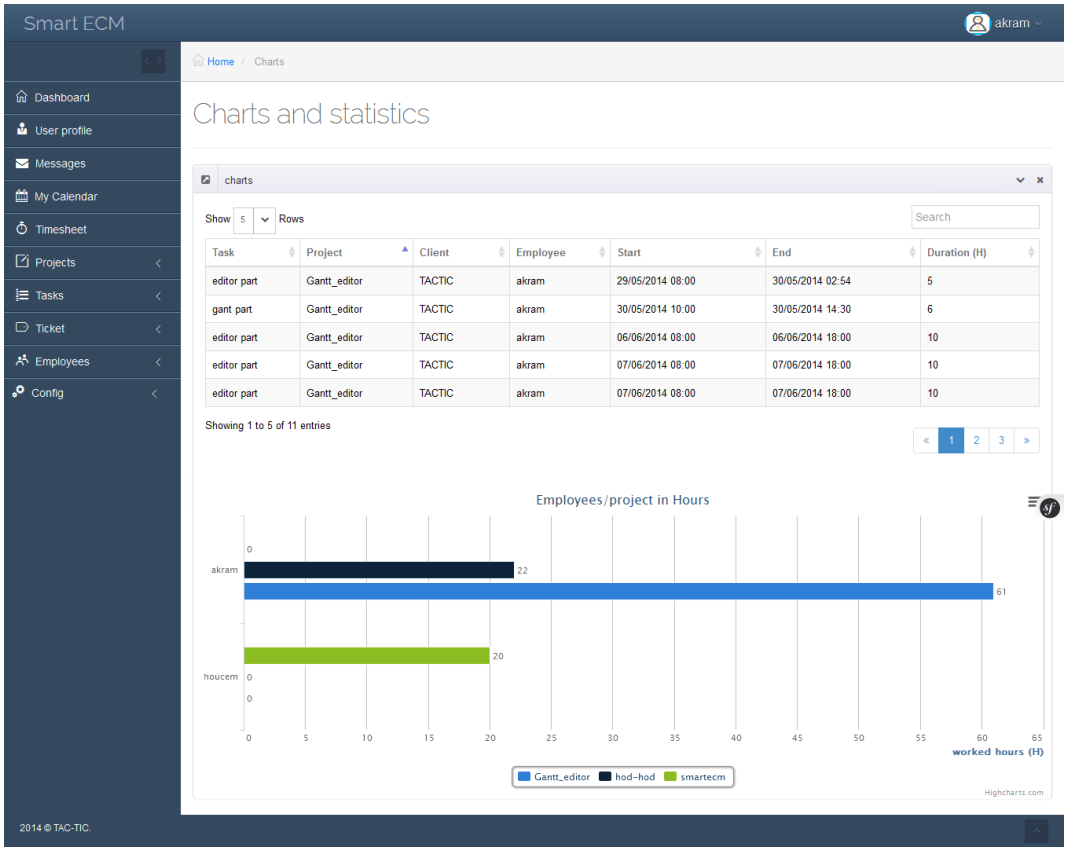


FIGURE 5.10 – statistiques sur la réalisation des projets/employés

5.4 Chronogramme

Ce projet a été réalisé du 10 février 2011 au 15 juin 2011. Nous avons tout d'abord commencé par une étude approfondie de l'existant pour pouvoir comprendre les besoins et débuter le travail. Ensuite, après avoir dégagé et compris ce qui est demandé, nous avons relevé les spécifications fonctionnelles et non fonctionnelles. Puis nous avons entamé la phase de conception et implémenté l'application demandée après avoir passé par une phase de documentation sur les technologies à utiliser. Dans un souci d'organisation et afin de mener à bien le projet, nous avons établi un chronogramme détaillé des différentes tâches qui constituaient le projet illustré par la figure suivante.

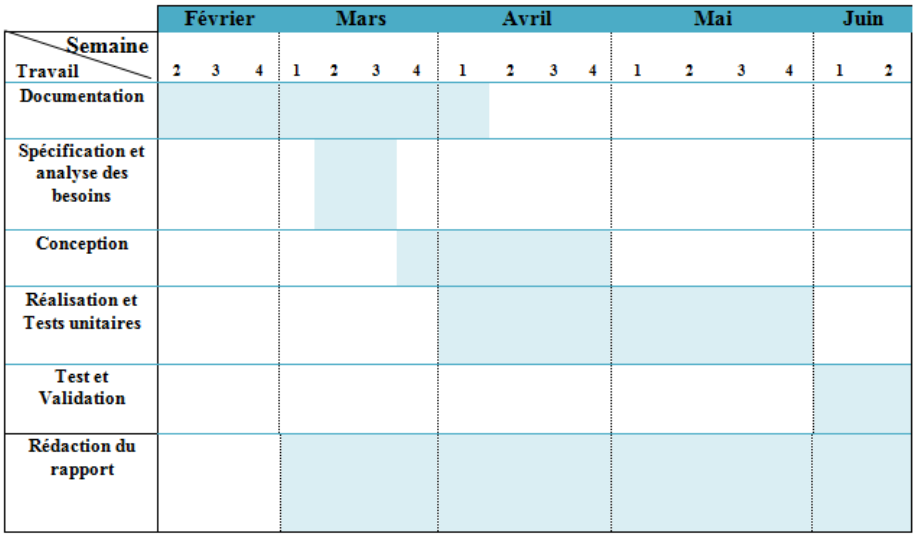


FIGURE 5.11 – Le chronogramme du travail

Conclusion

Dans ce chapitre, nous avons traité les détails de la réalisation de l'application moyennant le framework Symfony2. A la fin nous avons présenté le chronogramme modélisant les étapes de notre travail.

La partie suivante sera consacrée pour la conclusion générale et les perspectives de notre application

Conclusion Générale

le présent projet se situe avec l'objectif de réaliser une application web qu'on peut la considérer comme un outil B2B «business to business» de gestion de projet dès l'initiation de l'appel jusqu'à la clôture du projet et qui offre un nombre de fonctionnalités pour les différents utilisateurs. Durant ce projet, nous avons réalisé une étude claire et concise des plus importantes solutions existantes pour dégager les principaux critères et services offerts par ses dernières. Dans le troisième chapitre, nous avons entamé l'étape d'analyse et de spécification des besoins comme il a été convenu dans la méthodologie UML. Puis, nous avons fait une étude des technologies susceptibles d'être utiliser pour la réalisation de l'application. Le chapitre quatre aborde l'étape de conception globale et détaillée. Enfin, cette conception a été couronnée par la mise en œuvre à travers quelques interfaces.

Par ailleurs, la réalisation de cette application web nous a été très bénéfique sur le plan technique. En effet, nous avons eu une opportunité de mieux maitriser les languages PHP5, Javascript, CSS3 et découvrir les nouveaux outils de développement d'application Web et approfondir nos connaissances rudimentaires sur la plateforme symfony.

En conclusion, ce travail a accompli ses objectifs, mais ce n'est qu'un début d'un long processus ; il existe quelque extensions qui peuvent enrichir notre application comme l'ajout d'autres services, l'implémentation d'une méthode de gestion des risques et l'amélioration l'expérience utilisateur par l'ajout d'un système de notification en temps réel.

Bibliographie

[1] F. Vallée ROQUES, *UML 2 par la pratique*, Editions Eyrolls, 2005, 180p.

[2] C. Soutou. *UML 2 pour les bases des données*, Editions Eyrolls, 2005, 44p.

Netographie

[N1] : http ://cian.developpez.com/uml2/tutoriel/sequence/ le 23/03/2014

[N2] : http ://fr.wikipedia.org/wiki/Architecture_trois_tiers le 01/05/2014

[N3] : http ://fr.wikipedia.org/wiki/Modèle-Vue-Contrôleur le 01/05/2014

[N4] : http ://doc.ubuntu-fr.org/eclipse 01/03/2014.

[N5] : http ://symfony.com/fr/doc/current/quick_tour/the_architecture.html 01/03/2014.

Annexe A

L'architecture de Symfony2

Comprendre l'arborescence

L'arborescence d'une application Symfony2 est plutôt flexible mais celle de la distribution Standard Edition reflète la structure typique suivante :

- app/ : La configuration de l'application.
- src/ : Le code PHP du projet
- vendor/ : Les bibliothèques tierces
- web/ : Le répertoire Web racine.

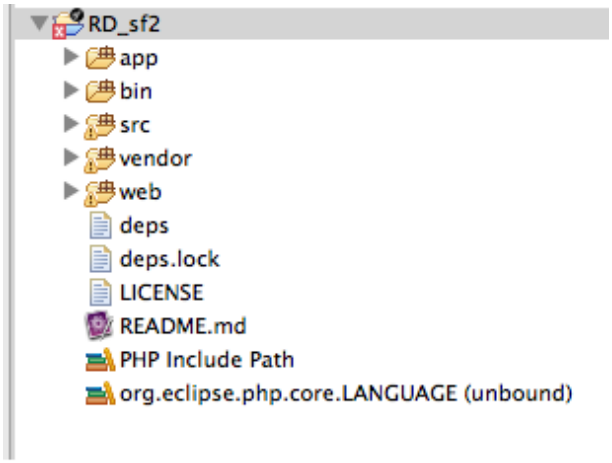


FIGURE A.1 – L'arborescence d'une application Symfony2

Le répertoire web/

Le répertoire Web racine est l'endroit ou se situent tous les fichiers statiques et publics comme les images, les feuilles de styles et les fichiers javascript. Il contient aussi tout contrôleur frontal :

```
1  <?php
2  // web/app.php
3  require_once __DIR__.'/../app/bootstrap.php.cache';
4  require_once __DIR__.'/../app/AppKernel.php';
5  use Symfony\Component\HttpFoundation\Request;
6  $kernel = new AppKernel('prod', false);
7  $kernel->loadClassCache();
8  $kernel->handle(Request::createFromGlobals())->send();
```

Le noyau (kernel) requiert d'abord le fichier bootstrap.php.cache, qui amorce le framework et l'autoloader. Comme tout contrôleur frontal, app.php utilise une classe Kernel AppKernel pour amorcer une application.

Le répertoire app/

La classe AppKernel est le point d'entrée principal de la configuration de l'application et, en tant que tel, il est placé dans le répertoire app/. Cette classe doit implémenter deux méthodes :

- registerBundles() doit retourner un tableau de tous les Bundles nécessaires au fonctionnement de l'application.
- registerContainerConfiguration() charge la configuration de l'application.

L'autoloading PHP peut être configuré via app/autoload.php :

```
1  <?php
2  // app/autoload.php
3  use Symfony\Component\ClassLoader\UniversalClassLoader;
4  $loader = new UniversalClassLoader();
5  $loader->registerNamespaces(array(
6      'Symfony'          => array(__DIR__.'/../vendor/symfony/symfony/src',
```

```
7                                 __DIR__.'/../vendor/bundles'),
8       'Sensio'           => __DIR__.'/../vendor/bundles',
9       'JMS'              => __DIR__.'/../vendor/jms/',
10      'Doctrine\\Common' => __DIR__.'/../vendor/doctrine/common/lib',
11      'Doctrine\\DBAL'   => __DIR__.'/../vendor/doctrine/dbal/lib',
12      'Doctrine'         => __DIR__.'/../vendor/doctrine/orm/lib',
13      'Monolog'          => __DIR__.'/../vendor/monolog/monolog/src',
14      'Assetic'          => __DIR__.'/../vendor/kriswallsmith/assetic/src',
15      'Metadata'         => __DIR__.'/../vendor/jms/metadata/src',
16  ));
17  $loader->registerPrefixes(array(
18      'Twig_Extensions_' => __DIR__.'/../vendor/twig/extensions/lib',
19      'Twig_'            => __DIR__.'/../vendor/twig/twig/lib',
20  ));
21  // ...
22  $loader->registerNamespaceFallbacks(array(
23      __DIR__.'/../src',
24  ));
25  $loader->register();
```

La classe UniversalClassLoader est utilisée pour charger automatiquement les fichiers qui respectent les standards d'interopérabilité technique pour les namespace PHP 5.3, ou les conventions de nommage PEAR pour les classes.

Comprendre le système de bundles

Un Bundle est un répertoire qui contient un ensemble de fichiers (classes PHP, feuilles de style, JavaScripts, images, ...) qui implémentent une fonctionnalité unique (un blog, un forum, etc). Ils nous donnent la flexibilité d'utiliser des fonctionnalités pré-construites dans des bundles tiers ou de distribuer nos propres bundles. Ils facilitent la synergie et le choix des fonctionnalités à activer pour notre application et les optimisent de la manière que nous désirons. Une application est constituée de bundles définis dans la méthode registerBundles() de la classe *AppKernel*.[N5]

Utilisation de bibliothèques externes (Vendors)

Il y a de fortes probabilités que votre application dépende de bibliothèques tierces. Celles-ci doivent être stockées dans le répertoire vendor/. Ce répertoire contient déjà les bibliothèques de Symfony2, la bibliothèque SwiftMailer, l'ORM Doctrine, le système de template Twig et d'autres bibliothèques et bundles.

Comprendre le Cache et les Logs

Symfony2 est probablement l'un des plus rapides framework full-stack existant. Mais comment peut-il être si rapide s'il analyse et interprète des dizaines de fichiers YAML et XML à chaque requête ? Cette rapidité est en partie due à son système de cache. La configuration de l'application est uniquement analysée lors de la première requête, puis compilée en PHP pur stocké dans le répertoire app/cache/ de l'application.

Principe de fonctionnement

c'est le contrôleur principal qui reçoit les requêtes http. Celles-ci sont identifiées par des URL comme /contact, /shop, etc. Ensuite, le kernel coordonne le tout. Dans Symfony, il y a un outil appelé le routing. Son rôle est d'associer une URL avec la fonction du contrôleur correspondante. C'est enfin au contrôleur à qui revient le rôle de se servir des informations de la requête pour envoyer la réponse appropriée.

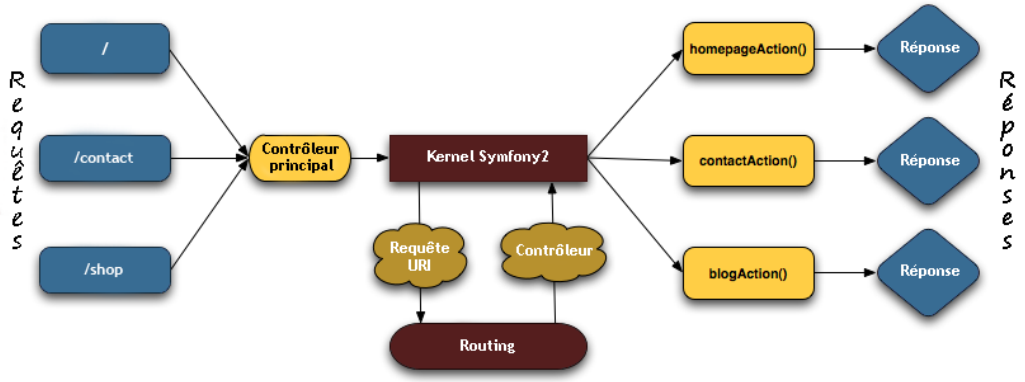


FIGURE A.2 – Schématisation du flux d'exécution d'une requête par Symfony

La web debug toolbar

Un des outils les plus indispensable en cours de développement avec Symfony2 est la web debug toolbar, plus communément appelée profiler.



FIGURE A.3 – Le toolbare de débougage

Cette barre nous offre pas mal d'informations dès le premier coup d'oeil.

- La version de php est les éventuelles extensions d'activées
- Les informations sur notre environnement.
- Quelques informations techniques de notre page
- Le temps d'exécution de la page.
- La quantité de mémoire utilisée pour la génération de notre page.
- Les informations sur notre utilisateur.
- Le nombre de requêtes exécutées en base de données.